RecoMart Final Assignment Submission Report

Student Name: Barath

Roll Number: _____

Course: Data Management for Machine Learning

Assignment Title: End-to-End Data Management Pipeline for a Recommendation System

Client: RecoMart

Project Root: C:\Users\barath\recomart-pipeline

Submission Date: 2026-04-29

Executive Summary

This submission presents an end-to-end recommendation data pipeline for RecoMart. The implementation covers problem formulation, multi-source ingestion, raw data storage, data validation with remediation and revalidation, preparation, feature engineering, a custom feature store, model tracking, and pipeline orchestration.

• Ingest at least two data types: CSV-based interaction/catalog files and API/raw JSON product data.

• Store source data in a structured local data lake partitioned by source and timestamp.

• Validate data quality, produce issue logs, auto-remediate failed datasets, and revalidate.

• Prepare clean datasets for EDA and downstream recommendation modeling.

• Implement feature engineering and a custom metadata-based feature store with versioned retrieval.

• Track model runs, parameters, and metrics using MLflow or equivalent metadata tracking.

• Orchestrate the full workflow end to end using Prefect.

1. Problem Formulation

RecoMart requires a scalable, maintainable, and automated data management pipeline to ingest fresh multi-source e-commerce data, validate and remediate quality issues, prepare and transform datasets, manage reusable recommendation features, and train or refresh recommendation models with orchestration and experiment tracking.

Expected Outputs

• Clean datasets for EDA.

• Engineered features for collaborative/content-based recommendation models.

• Deployable recommendation model artifacts and inference-ready datasets.

• Operational logs, validation reports, and tracked training metadata.

Evaluation Metrics

• Precision@K

• Recall@K

• NDCG@K

2. Data Collection and Ingestion

The pipeline ingests at least two data types, satisfying the assignment requirement: CSV-based RetailRocket interaction/catalog data and JSON-based DummyJSON product/category data.

Discovered Ingestion Files

Source Key	Exact Path	Exists
events_csv	C:\Users\barath\recomart-pipeline\data\raw\retailrocket\load_date=2026-04-28\load_hour=14\events.csv	Yes
category_tree_csv	C:\Users\barath\recomart-pipeline\data\raw\retailrocket\load_date=2026-04-26\load_hour=19\category_tree.csv	Yes
item_properties_part1_csv	C:\Users\barath\recomart-pipeline\data\raw\retailrocket\load_date=2026-04-28\load_hour=14\item_properties_part1	Yes
item_properties_part2_csv	C:\Users\barath\recomart-pipeline\data\raw\retailrocket\load_date=2026-04-28\load_hour=14\item_properties_part2	Yes
products_raw_json	C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=2026-04-28\load_hour=14\products_raw.json	Yes
categories_raw_json	C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=2026-04-28\load_hour=14\categories_raw.json	Yes
products_parquet	C:\Users\barath\recomart-pipeline\data\bronze\dummyjson\products.parquet	Yes
categories_parquet	C:\Users\barath\recomart-pipeline\data\bronze\dummyjson\categories.parquet	Yes

3. Raw Data Storage

Exact Storage Layout

Folder	Purpose	Exists
C:\Users\barath\recomart-pipeline\data\raw	Raw source data	Yes
C:\Users\barath\recomart-pipeline\data\bronze	Structured bronze datasets	Yes
C:\Users\barath\recomart-pipeline\data\remediated\raw	Remediated raw outputs	Yes
C:\Users\barath\recomart-pipeline\data\remediated\bronze	Remediated bronze outputs	Yes
C:\Users\barath\recomart-pipeline\reports\validation	Validation reports	Yes
C:\Users\barath\recomart-pipeline\logs	Execution logs	Yes

4. Data Profiling and Validation

The validation stage includes robust project-root discovery, issue logging, remediation, revalidation, and PDF/JSON/CSV report generation.

Validation Run Summary

Field	Value
Validation Run ID	20260428T144031Z
Initial Status	FAIL
Final Status	PASS
JSON Report	C:\Users\barath\recomart-pipeline\reports\validation\data_quality_report_20260428T144031Z.json
PDF Report	C:\Users\barath\recomart-pipeline\reports\validation\data_quality_report_20260428T144031Z.pdf
Fix Log	C:\Users\barath\recomart-pipeline\reports\validation\fix_log_20260428T144031Z.csv

Validation Artifacts

Artifact	Path	Exists
initial_issues	C:\Users\barath\recomart-pipeline\reports\validation\validation_issues_initial_20260428T144031Z.csv	Yes
initial_summary	C:\Users\barath\recomart-pipeline\reports\validation\validation_summary_initial_20260428T144031Z.csv	Yes
revalidation_issues	C:\Users\barath\recomart-pipeline\reports\validation\validation_issues_revalidated_20260428T144031Z.csv	Yes
revalidation_summary	C:\Users\barath\recomart-pipeline\reports\validation\validation_summary_revalidated_20260428T144031Z.csv	Yes
fix_log	C:\Users\barath\recomart-pipeline\reports\validation\fix_log_20260428T144031Z.csv	Yes
json_report	C:\Users\barath\recomart-pipeline\reports\validation\data_quality_report_20260428T144031Z.json	Yes
pdf_report	C:\Users\barath\recomart-pipeline\reports\validation\data_quality_report_20260428T144031Z.pdf	Yes
validation_log	C:\Users\barath\recomart-pipeline\logs\validation_log_20260428T144031Z.json	Yes

5. Data Preparation

Prepared datasets are created after cleaning and remediation. Based on the validation pipeline, this includes fixing problematic raw and bronze files and carrying forward standardized datasets for downstream use.

• Missing/invalid values addressed during remediation.

• Dataset consistency rechecked in revalidation.

• Prepared outputs routed into remediated/raw and remediated/bronze folders.

6. Feature Engineering and Transformation

Feature engineering logic is expected to transform prepared recommendation data into reusable training and inference features. The repository structure indicates a dedicated feature stage under src and a separate feature_store area for metadata-based retrieval.

• Recommendation-oriented transformation stage exists under src.

• Feature outputs are intended to be version-aware and retrievable for training/inference.

• Supporting code is included in the appendix automatically from src/ and feature_store/.

7. Feature Store

Feature Store Metadata

Item	Value
Feature Store Root	C:\Users\barath\recomart-pipeline\feature_store
Registry File	C:\Users\barath\recomart-pipeline\feature_store\registry\feature_registry.yaml
Registry Present	Yes

8. Data Versioning and Lineage

• Raw data is partitioned by source plus timestamp folders such as load_date and load_hour.

• Validation outputs are versioned by run ID in reports/validation.

• Logs are timestamped and stored separately for auditability.

• Remediated outputs preserve lineage from original failed datasets to corrected datasets.

9. Model Training and Evaluation

The assignment requires tracked model metadata, parameters, and metrics. This report attempts to auto-discover those details from the local mlruns directory.

MLflow Discovery Summary

Field	Value
experiment_name	recomart_content_based
best_run_id	5f6e8cef85a144a282e26c5ebb82690e
best_metric_name	catalog_size_used
best_metric_value	0.0
artifact_uri	file:///C:/Users/barath/recomart-pipeline/mlruns/789510746751042322/5f6e8cef85a144a282e26c5ebb8269
status_note	MLflow run and metric discovered from local mlruns folder.

10. Pipeline Orchestration

The assignment requires orchestration through Prefect, Airflow, or Dagster. This report auto-discovers the latest Prefect execution folder under runs/prefect.

Prefect Discovery Summary

Field	Value
flow_name	20260429_164131
latest_run_folder	C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131
executed_notebooks_folder	C:\Users\barath\recomart-pipeline\runs\prefect\20260429_164131\executed_notebooks
status_note	Latest Prefect run folder discovered successfully.

11. Code Organization

The assignment asks for source code organized by pipeline stage. The script below auto-lists current stage folders under src/.

Detected src/ Stage Folders

Detected Stage Folder
01-ingestion
02-validation
03-eda-prep
04-features
05-training
06-orchestration
07-submission

Repository Snapshot

```
src/
    01-ingestion/
        dummyjson_ingestion.ipynb
        retailrocket_ingestion.ipynb
    02-validation/
        run_validation.ipynb
        run_validation.txt
    03-eda-prep/
        eda_prep copy 2.txt
        eda_prep-copy.ipynb
    04-features/
        materialize_features.ipynb
        registry_loader copy.ipynb
        registry_loader.ipynb
    05-training/
        comparision.ipynb
        content-based-training.ipynb
        item-based-collaborative.ipynb
        svd.ipynb
    06-orchestration/
        pipeline_orchestration_prefect.ipynb
    07-submission/
        final_submission.ipynb
```

12. Submission Checklist

• Problem definition, objectives, sources, and outputs documented.

• Ingestion evidence included for CSV and JSON sources.

• Structured raw/bronze/remediated/report/log folders documented.

• Validation report evidence included with exact run ID and artifacts.

• Feature store path and registry location documented.

• MLflow section included and auto-populated when mlruns is present.

• Prefect orchestration section included and auto-populated when runs/prefect is present.

• Code appendix included from src/ and feature_store/.

• Separate video walkthrough still required for final submission package.

Appendix A: Selected Code Listings

Code: feature_store\registry\feature_registry.yaml

```
registry_name: recomart_feature_registry
registry_version: v1
created_for: recommendation_pipeline
description: Custom metadata registry for training and inference features

entities:
  - name: user
    join_key: user_id
    description: Unique customer identifier from interaction logs

  - name: item
    join_key: item_id
    description: Unique product/item identifier from interaction logs or metadata

  - name: user_item
    join_keys: [user_id, item_id]
    description: Composite user-item key for pairwise interaction features

feature_sets:
  - name: user_features
    entity: user
    version: v1
    source_table: interactions_prepared.csv
    storage_path: data/featured/v1/user_features.parquet
    serving_modes: [training, inference]
    features:
      - name: user_total_interactions
        dtype: int
        source_column: user_id
        transformation: count of all interactions per user
        freshness: daily

      - name: user_view_count
        dtype: int
        source_column: event
        transformation: count where event == view
        freshness: daily

      - name: user_addtocart_count
        dtype: int
        source_column: event
        transformation: count where event == addtocart
        freshness: daily

      - name: user_transaction_count
        dtype: int
        source_column: event
        transformation: count where event == transaction
        freshness: daily

      - name: user_avg_event_weight
        dtype: float
        source_column: event_weight
        transformation: mean event_weight grouped by user_id
        freshness: daily

      - name: user_active_days
        dtype: int
        source_column: event_date
        transformation: distinct active dates per user
        freshness: daily

      - name: user_last_seen_recency_days
        dtype: float
        source_column: event_ts
        transformation: days since most recent interaction
        freshness: daily

  - name: item_features
    entity: item
    version: v1
    source_table: interactions_prepared.csv + products_prepared.csv
    storage_path: data/featured/v1/item_features.parquet
    serving_modes: [training, inference]
    features:
      - name: item_total_interactions
        dtype: int
        source_column: item_id
        transformation: count of all interactions per item
        freshness: daily

      - name: item_view_count
        dtype: int
        source_column: event
        transformation: count where event == view grouped by item_id
        freshness: daily
```

```
      - name: item_addtocart_count
        dtype: int
        source_column: event
        transformation: count where event == addtocart grouped by item_id
        freshness: daily

      - name: item_transaction_count
        dtype: int
        source_column: event
        transformation: count where event == transaction grouped by item_id
        freshness: daily

      - name: item_avg_event_weight
        dtype: float
        source_column: event_weight
        transformation: mean event_weight grouped by item_id
        freshness: daily

      - name: item_popularity_rank
        dtype: int
        source_column: item_total_interactions
        transformation: descending rank by interaction count
        freshness: daily

      - name: price_norm
        dtype: float
        source_column: price_norm
        transformation: min-max normalized price from product metadata
        freshness: daily

      - name: rating_norm
        dtype: float
        source_column: rating_norm
        transformation: min-max normalized rating from product metadata
        freshness: daily

      - name: stock_norm
        dtype: float
        source_column: stock_norm
        transformation: min-max normalized stock from product metadata
        freshness: daily

      - name: category
        dtype: string
        source_column: category
        transformation: standardized lowercase category label
        freshness: daily

  - name: interaction_features
    entity: user_item
    version: v1
    source_table: interactions_prepared.csv
    storage_path: data/featured/v1/interaction_features.parquet
    serving_modes: [training]
    features:
    - name: user_item_event_weight_sum
      dtype: float
      source_column: event_weight
      transformation: sum event_weight grouped by user_id and item_id
      freshness: daily

    - name: user_item_interaction_count
      dtype: int
      source_column: item_id
      transformation: count of interactions grouped by user_id and item_id
      freshness: daily

    - name: user_item_last_event_recency_days
      dtype: float
      source_column: event_ts
      transformation: days since latest interaction for user-item pair
      freshness: daily

governance:
  owner: recomart_data_platform
  lineage:
    raw_sources:
      - data/raw/retailrocket/events.csv
      - data/raw/dummyjson/products_raw.json
    prepared_sources:
      - data/prepared/interactions_prepared.csv
      - data/prepared/products_prepared.csv
  retrieval_rules:
    training:
      include_feature_sets: [user_features, item_features, interaction_features]
    inference:
      include_feature_sets: [user_features, item_features]
```

Code: src\01-ingestion\dummyjson_ingestion.ipynb

```
# DM4ML -Assignment - Ingestion - API

import json
import time
import random
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import requests

# ============================================================
# RecoMart - DummyJSON API Ingestion Script
# - Fetches product + category data from DummyJSON
# - Retries on transient failures
# - Falls back to embedded sample data if API is unreachable
# - Stores raw JSON, bronze parquet, logs, and manifest
# ============================================================

# ------------------------------
# 1) CONFIG
# ------------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "dummyjson"

USE_OFFLINE_FALLBACK = True
MAX_ATTEMPTS = 5
REQUEST_TIMEOUT = 30
PAGE_SIZE = 30

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

BASE_URL = "https://dummyjson.com"
PRODUCTS_ENDPOINT = f"{BASE_URL}/products"
CATEGORIES_ENDPOINT = f"{BASE_URL}/products/categories"

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" / f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE_NAME}_ingestion_log.jsonl"
MANIFEST_FILE = METADATA_DIR / f"{SOURCE_NAME}_manifest_{BATCH_ID}.json"

# ------------------------------
# 2) LOGGING
# ------------------------------
def log_event(stage, status, message, extra=None):
    record = {
        "event_ts": datetime.now(timezone.utc).isoformat(),
        "source": SOURCE_NAME,
        "stage": stage,
        "status": status,
        "message": message,
        "batch_id": BATCH_ID,
    }
    if extra:
        record.update(extra)

    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")

# ------------------------------
# 3) OFFLINE FALLBACK DATA
# ------------------------------
def get_fallback_products():
    return {
        "products": [
            {
                "id": 1,
                "title": "Essence Mascara Lash Princess",
                "description": "Lengthening mascara for daily wear",
                "category": "beauty",
                "price": 9.99,
                "discountPercentage": 7.17,
                "rating": 4.94,
                "stock": 5,
                "brand": "Essence",
                "sku": "BEAUTY-001",
                "weight": 20,
                "dimensions": {"width": 2.0, "height": 12.0, "depth": 2.0},
```

```
                "warrantyInformation": "6 months",
                "shippingInformation": "Ships in 2 days",
                "availabilityStatus": "Low Stock",
                "reviews": [],
                "returnPolicy": "7 days return",
                "minimumOrderQuantity": 1,
            },
            {
                "id": 2,
                "title": "iPhone 15 Case",
                "description": "Protective slim case",
                "category": "smartphones",
                "price": 19.99,
                "discountPercentage": 5.0,
                "rating": 4.5,
                "stock": 50,
                "brand": "AppleGear",
                "sku": "PHONE-002",
                "weight": 50,
                "dimensions": {"width": 8.0, "height": 15.0, "depth": 1.0},
                "warrantyInformation": "12 months",
                "shippingInformation": "Ships in 1 day",
                "availabilityStatus": "In Stock",
                "reviews": [],
                "returnPolicy": "14 days return",
                "minimumOrderQuantity": 1,
            },
            {
                "id": 3,
                "title": "Running Shoes",
                "description": "Comfortable sports shoes",
                "category": "mens-shoes",
                "price": 79.99,
                "discountPercentage": 10.0,
                "rating": 4.3,
                "stock": 22,
                "brand": "SprintX",
                "sku": "SHOE-003",
                "weight": 700,
                "dimensions": {"width": 12.0, "height": 10.0, "depth": 32.0},
                "warrantyInformation": "3 months",
                "shippingInformation": "Ships in 3 days",
                "availabilityStatus": "In Stock",
                "reviews": [],
                "returnPolicy": "10 days return",
                "minimumOrderQuantity": 1,
            },
        ],
        "total": 3,
        "skip": 0,
        "limit": 30,
    }

def get_fallback_categories():
    return ["beauty", "smartphones", "mens-shoes"]

# ----------------------------
# 4) HTTP FETCH
# ----------------------------
def fetch_json(url, params=None, timeout=30, max_attempts=5):
    last_error = None

    headers = {
        "User-Agent": "RecoMart-Assignment/1.0",
        "Accept": "application/json",
        "Connection": "close",
    }

    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.get(url, params=params, headers=headers, timeout=timeout)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            last_error = str(e)

            log_event(
                "api_request",
                "failed",
                f"Request failed on attempt {attempt}",
                extra={"url": url, "params": params, "error": last_error},
            )

            if attempt < max_attempts:
                sleep_seconds = (2 ** attempt) + random.uniform(0.5, 1.5)
                time.sleep(sleep_seconds)

    raise RuntimeError(f"API request failed after {max_attempts} attempts: {last_error}")

def fetch_all_products(page_size=30):
    log_event("api_fetch_products", "started", "Fetching products from DummyJSON")

    first_page = fetch_json(PRODUCTS_ENDPOINT, params={"limit": page_size, "skip": 0}, timeout=REQUEST_TIMEOUT, max_attempts=MAX_A
```

```
    products = first_page.get("products", [])
    total = first_page.get("total", len(products))

    all_products = list(products)
    skip = len(products)

    while skip < total:
        page = fetch_json(PRODUCTS_ENDPOINT, params={"limit": page_size, "skip": skip}, timeout=REQUEST_TIMEOUT, max_attempts=MAX_A
        page_products = page.get("products", [])
        all_products.extend(page_products)

        log_event(
            "api_fetch_products_page",
            "success",
            f"Fetched page with skip={skip}",
            extra={"page_count": len(page_products), "running_total": len(all_products)},
        )

        skip += page_size
        time.sleep(1)

    log_event(
        "api_fetch_products",
        "success",
        "Fetched all product pages successfully",
        extra={"total_products": len(all_products)},
    )

    return {"products": all_products, "total": len(all_products)}

def fetch_categories():
    log_event("api_fetch_categories", "started", "Fetching categories from DummyJSON")
    categories = fetch_json(CATEGORIES_ENDPOINT, timeout=REQUEST_TIMEOUT, max_attempts=MAX_ATTEMPTS)
    log_event(
        "api_fetch_categories",
        "success",
        "Fetched categories successfully",
        extra={"category_count": len(categories) if isinstance(categories, list) else 0},
    )
    return categories

# ----------------------------
# 5) HELPERS
# ----------------------------
def save_raw_json(obj, output_path):
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2, ensure_ascii=False)

def add_ingestion_metadata(df, source_file, source_mode):
    out = df.copy()
    out["source_system"] = SOURCE_NAME
    out["source_file"] = source_file
    out["source_mode"] = source_mode
    out["batch_id"] = BATCH_ID
    out["ingestion_ts"] = INGESTION_TS
    return out

def categories_to_dataframe(categories):
    if not isinstance(categories, list):
        return pd.DataFrame(columns=["category"])

    if len(categories) == 0:
        return pd.DataFrame(columns=["category"])

    if isinstance(categories[0], dict):
        return pd.json_normalize(categories)
    return pd.DataFrame({"category": categories})

# ----------------------------
# 6) MAIN
# ----------------------------
def main():
    manifest = {
        "source": SOURCE_NAME,
        "batch_id": BATCH_ID,
        "ingestion_ts": INGESTION_TS,
        "raw_output_dir": str(RAW_DIR),
        "bronze_output_dir": str(BRONZE_DIR),
        "files": [],
    }

    source_mode = "api"

    try:
        products_payload = fetch_all_products(page_size=PAGE_SIZE)
        categories_payload = fetch_categories()
    except Exception as e:
        log_event("api_ingestion", "failed", "Live API ingestion failed", extra={"error": str(e)})

        if not USE_OFFLINE_FALLBACK:
            raise

        source_mode = "offline_fallback"
```

```
    products_payload = get_fallback_products()
    categories_payload = get_fallback_categories()

    log_event(
        "api_ingestion",
        "fallback",
        "Switched to offline fallback payload",
        extra={"fallback_products": len(products_payload.get('products', []))},
    )

products_raw_path = RAW_DIR / "products_raw.json"
categories_raw_path = RAW_DIR / "categories_raw.json"

save_raw_json(products_payload, products_raw_path)
save_raw_json(categories_payload, categories_raw_path)

manifest["files"].append(
    {
        "file_name": "products_raw.json",
        "raw_path": str(products_raw_path),
        "record_count": len(products_payload.get("products", [])),
        "source_mode": source_mode,
        "saved_at": datetime.now(timezone.utc).isoformat(),
    }
)

manifest["files"].append(
    {
        "file_name": "categories_raw.json",
        "raw_path": str(categories_raw_path),
        "record_count": len(categories_payload) if isinstance(categories_payload, list) else 0,
        "source_mode": source_mode,
        "saved_at": datetime.now(timezone.utc).isoformat(),
    }
)

products = products_payload.get("products", [])
products_df = pd.json_normalize(products, sep="_")
products_df = add_ingestion_metadata(products_df, "products_raw.json", source_mode)

categories_df = categories_to_dataframe(categories_payload)
categories_df = add_ingestion_metadata(categories_df, "categories_raw.json", source_mode)

products_parquet = BRONZE_DIR / "products.parquet"
categories_parquet = BRONZE_DIR / "categories.parquet"
summary_csv = BRONZE_DIR / "ingestion_summary.csv"

products_df.to_parquet(products_parquet, index=False)
categories_df.to_parquet(categories_parquet, index=False)

summary_df = pd.DataFrame(
    [
        {
            "dataset": "products",
            "rows": len(products_df),
            "columns": len(products_df.columns),
            "output_path": str(products_parquet),
            "source_mode": source_mode,
        },
        {
            "dataset": "categories",
            "rows": len(categories_df),
            "columns": len(categories_df.columns),
            "output_path": str(categories_parquet),
            "source_mode": source_mode,
        },
    ]
)
summary_df.to_csv(summary_csv, index=False)

manifest["bronze_outputs"] = [
    str(products_parquet),
    str(categories_parquet),
    str(summary_csv),
]
manifest["source_mode"] = source_mode

with open(MANIFEST_FILE, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2)

log_event(
    "pipeline",
    "success",
    "DummyJSON ingestion completed successfully",
    extra={
        "source_mode": source_mode,
        "manifest_file": str(MANIFEST_FILE),
        "products_rows": len(products_df),
        "categories_rows": len(categories_df),
    },
)

print("DummyJSON ingestion completed successfully")
```

```
    print(f"Source mode: {source_mode}")
    print(f"Raw dir: {RAW_DIR}")
    print(f"Bronze dir: {BRONZE_DIR}")
    print(f"Manifest: {MANIFEST_FILE}")
    print("\nSummary:")
    print(summary_df)

if __name__ == "__main__":
    main()
```

Code: src\01-ingestion\retailrocket_ingestion.ipynb

```
# DM4ML -Assignment - Ingestion - File

# ============================================================
# Retailrocket Ingestion Notebook / Script
# Purpose:
#   - Download Retailrocket dataset via kagglehub
#   - Copy raw files into partitioned local data lake folders
#   - Log success/failure events
#   - Create a manifest for audit/lineage
#   - Build bronze parquet datasets for downstream use
# ============================================================

# ----------------------------
# 0) Optional: install packages
# Uncomment if needed in Jupyter
# ----------------------------
# import sys
# !{sys.executable} -m pip install kagglehub pandas pyarrow

import json
import shutil
import time
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import kagglehub

# ============================
# 1) CONFIG
# ============================
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "retailrocket"
DATASET_ID = "retailrocket/ecommerce-dataset"

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" / f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE_NAME}_ingestion_log.jsonl"
MANIFEST_FILE = METADATA_DIR / f"{SOURCE_NAME}_manifest_{BATCH_ID}.json"

EXPECTED_FILES = [
    "events.csv",
    "category_tree.csv",
    "item_properties_part1.csv",
    "item_properties_part2.csv",
]

# ============================
# 2) LOGGING
# ============================
def log_event(stage, status, message, extra=None):
    record = {
        "event_ts": datetime.now(timezone.utc).isoformat(),
        "source": SOURCE_NAME,
        "stage": stage,
        "status": status,
        "message": message,
        "batch_id": BATCH_ID,
    }
    if extra:
        record.update(extra)
```

```
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")

# ============================
# 3) RETRY WRAPPER
# ============================
def retry_download(dataset_id, max_attempts=3, wait_seconds=5):
    last_error = None
    for attempt in range(1, max_attempts + 1):
        try:
            log_event(
                stage="download",
                status="started",
                message=f"Download attempt {attempt} started",
                extra={"dataset_id": dataset_id},
            )
            path = kagglehub.dataset_download(dataset_id)
            log_event(
                stage="download",
                status="success",
                message=f"Download attempt {attempt} succeeded",
                extra={"dataset_path": str(path)},
            )
            return Path(path)
        except Exception as e:
            last_error = str(e)
            log_event(
                stage="download",
                status="failed",
                message=f"Download attempt {attempt} failed",
                extra={"error": last_error},
            )
            if attempt < max_attempts:
                time.sleep(wait_seconds)

    raise RuntimeError(f"Download failed after {max_attempts} attempts. Last error: {last_error}")

# ============================
# 4) DOWNLOAD DATASET
# ============================
dataset_path = retry_download(DATASET_ID, max_attempts=3, wait_seconds=5)
print("Downloaded dataset to:", dataset_path)

# ============================
# 5) DISCOVER FILES
# ============================
all_files = [p for p in dataset_path.rglob("*") if p.is_file()]
file_lookup = {p.name: p for p in all_files}

missing_files = [f for f in EXPECTED_FILES if f not in file_lookup]
if missing_files:
    log_event(
        stage="file_validation",
        status="failed",
        message="Expected files missing",
        extra={"missing_files": missing_files},
    )
    raise FileNotFoundError(f"Missing expected files: {missing_files}")

log_event(
    stage="file_validation",
    status="success",
    message="All expected files found",
    extra={"found_files": EXPECTED_FILES},
)

# ============================
# 6) COPY RAW FILES TO DATA LAKE
# ============================
manifest = {
    "source": SOURCE_NAME,
    "dataset_id": DATASET_ID,
    "batch_id": BATCH_ID,
    "ingestion_ts": INGESTION_TS,
    "raw_output_dir": str(RAW_DIR),
    "files": [],
}

for file_name in EXPECTED_FILES:
    src = file_lookup[file_name]
    dest = RAW_DIR / file_name
    shutil.copy2(src, dest)

    file_record = {
        "file_name": file_name,
        "source_path": str(src),
        "raw_path": str(dest),
        "copied_at": datetime.now(timezone.utc).isoformat(),
        "file_size_bytes": dest.stat().st_size,
    }
    manifest["files"].append(file_record)

log_event(
```

```
    stage="raw_storage",
    status="success",
    message="Raw files copied successfully",
    extra={"raw_dir": str(RAW_DIR)},
)

with open(MANIFEST_FILE, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2)

log_event(
    stage="manifest",
    status="success",
    message="Manifest created",
    extra={"manifest_file": str(MANIFEST_FILE)},
)

print("Raw files copied to:", RAW_DIR)
print("Manifest saved to:", MANIFEST_FILE)

# ============================
# 7) LOAD RAW CSVs
# ============================
events_df = pd.read_csv(RAW_DIR / "events.csv")
category_tree_df = pd.read_csv(RAW_DIR / "category_tree.csv")
item_properties_1_df = pd.read_csv(RAW_DIR / "item_properties_part1.csv")
item_properties_2_df = pd.read_csv(RAW_DIR / "item_properties_part2.csv")

item_properties_df = pd.concat(
    [item_properties_1_df, item_properties_2_df],
    ignore_index=True,
)

log_event(
    stage="raw_read",
    status="success",
    message="Raw CSV files loaded into pandas",
    extra={
        "events_rows": int(len(events_df)),
        "category_tree_rows": int(len(category_tree_df)),
        "item_properties_rows": int(len(item_properties_df)),
    },
)

# ============================
# 8) LIGHT BRONZE STANDARDIZATION
# ============================
def add_ingestion_metadata(df, source_file):
    out = df.copy()
    out["source_system"] = SOURCE_NAME
    out["source_file"] = source_file
    out["batch_id"] = BATCH_ID
    out["ingestion_ts"] = INGESTION_TS
    return out

events_bronze = add_ingestion_metadata(events_df, "events.csv")
category_tree_bronze = add_ingestion_metadata(category_tree_df, "category_tree.csv")
item_properties_bronze = add_ingestion_metadata(item_properties_df, "item_properties_combined.csv")

if "timestamp" in events_bronze.columns:
    events_bronze["event_ts"] = pd.to_datetime(events_bronze["timestamp"], unit="ms", errors="coerce")

if "timestamp" in item_properties_bronze.columns:
    item_properties_bronze["property_ts"] = pd.to_datetime(
        item_properties_bronze["timestamp"], unit="ms", errors="coerce"
    )

# ============================
# 9) WRITE BRONZE DATASETS
# ============================
events_out = BRONZE_DIR / "events.parquet"
category_tree_out = BRONZE_DIR / "category_tree.parquet"
item_properties_out = BRONZE_DIR / "item_properties.parquet"
summary_out = BRONZE_DIR / "ingestion_summary.csv"

events_bronze.to_parquet(events_out, index=False)
category_tree_bronze.to_parquet(category_tree_out, index=False)
item_properties_bronze.to_parquet(item_properties_out, index=False)

summary_df = pd.DataFrame(
    [
        {
            "dataset": "events",
            "rows": len(events_bronze),
            "columns": len(events_bronze.columns),
            "output_path": str(events_out),
        },
        {
            "dataset": "category_tree",
            "rows": len(category_tree_bronze),
            "columns": len(category_tree_bronze.columns),
            "output_path": str(category_tree_out),
        },
        {
```

```
            "dataset": "item_properties",
            "rows": len(item_properties_bronze),
            "columns": len(item_properties_bronze.columns),
            "output_path": str(item_properties_out),
        },
    ]
)

summary_df.to_csv(summary_out, index=False)

log_event(
    stage="bronze_write",
    status="success",
    message="Bronze datasets written successfully",
    extra={
        "events_out": str(events_out),
        "category_tree_out": str(category_tree_out),
        "item_properties_out": str(item_properties_out),
        "summary_out": str(summary_out),
    },
)

# ============================
# 10) PREVIEW
# ============================
print("\n=== Ingestion Summary ===")
print(summary_df)

print("\n=== Events Sample ===")
print(events_bronze.head())

print("\n=== Category Tree Sample ===")
print(category_tree_bronze.head())

print("\n=== Item Properties Sample ===")
print(item_properties_bronze.head())

log_event(
    stage="pipeline",
    status="success",
    message="Retailrocket ingestion pipeline completed successfully",
)
```

Code: src\02-validation\run_validation.ipynb

```
# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report

import json
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, PageBreak

# ============================================================
# RecoMart Validation - robust file discovery + validation
# with remediation, revalidation, and PDF reporting
# ============================================================

PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
BRONZE_ROOT = PROJECT_ROOT / "data" / "bronze"

REMEDIATED_RAW_ROOT = PROJECT_ROOT / "data" / "remediated" / "raw"
REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / "data" / "remediated" / "bronze"

REPORT_ROOT = PROJECT_ROOT / "reports" / "validation"
LOG_ROOT = PROJECT_ROOT / "logs"

for folder in [REPORT_ROOT, LOG_ROOT, REMEDIATED_RAW_ROOT, REMEDIATED_BRONZE_ROOT]:
    folder.mkdir(parents=True, exist_ok=True)

UTC_NOW = datetime.now(timezone.utc)
RUN_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
RUN_TS = UTC_NOW.isoformat()

LOG_FILE = LOG_ROOT / f"validation_log_{RUN_ID}.jsonl"
INITIAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_initial_{RUN_ID}.csv"
INITIAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_initial_{RUN_ID}.csv"
FINAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_revalidated_{RUN_ID}.csv"
FINAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_revalidated_{RUN_ID}.csv"
FIX_LOG_FILE = REPORT_ROOT / f"fix_log_{RUN_ID}.csv"
REPORT_FILE = REPORT_ROOT / f"data_quality_report_{RUN_ID}.json"
```

```
PDF_REPORT_FILE = REPORT_ROOT / f"data_quality_report_{RUN_ID}.pdf"

ALLOWED_EVENT_TYPES = {"view", "addtocart", "transaction"}


def log_event(stage, status, message, extra=None):
    record = {
        "event_ts": datetime.now(timezone.utc).isoformat(),
        "stage": stage,
        "status": status,
        "message": message,
        "run_id": RUN_ID,
    }
    if extra:
        record.update(extra)
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")


def add_issue(issues, dataset, check_type, severity, issue_count, description, file_path=None, phase="initial"):
    issues.append({
        "phase": phase,
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path) if file_path else None,
    })


def add_fix(fixes, dataset, action, rows_before=None, rows_after=None, details=None, file_path=None):
    fixes.append({
        "dataset": dataset,
        "action": action,
        "rows_before": rows_before,
        "rows_after": rows_after,
        "details": details,
        "file_path": str(file_path) if file_path else None,
    })


def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)


def missing_columns(df, expected_columns):
    return [c for c in expected_columns if c not in df.columns]


def duplicate_count(df, subset=None):
    return int(df.duplicated(subset=subset).sum())


def null_summary(df):
    result = {}
    for c in df.columns:
        n = int(df[c].isna().sum())
        if n > 0:
            result[c] = n
    return result


def invalid_numeric_range_count(df, column, min_value=None, max_value=None):
    if column not in df.columns:
        return None
    s = pd.to_numeric(df[column], errors="coerce")
    mask = pd.Series(False, index=df.index)
    if min_value is not None:
        mask = mask | (s < min_value)
    if max_value is not None:
        mask = mask | (s > max_value)
    return int(mask.fillna(False).sum())


def invalid_membership_count(df, column, allowed_values):
    if column not in df.columns:
        return None
    return int((~df[column].isin(allowed_values)).sum())


def invalid_timestamp_count(df, column, unit=None):
    if column not in df.columns:
        return None
    parsed = pd.to_datetime(df[column], errors="coerce", unit=unit)
    return int(parsed.isna().sum())

def build_dataset_summary(dataset_name, df, issues, phase):
```

```
    dataset_issues = [x for x in issues if x["dataset"] == dataset_name and x["phase"] == phase]
    error_count = sum(x["issue_count"] for x in dataset_issues if x["severity"] == "error")
    warning_count = sum(x["issue_count"] for x in dataset_issues if x["severity"] == "warning")
    return {
        "phase": phase,
        "dataset": dataset_name,
        "rows": int(len(df)) if df is not None else 0,
        "columns": int(len(df.columns)) if df is not None else 0,
        "errors": int(error_count),
        "warnings": int(warning_count),
        "status": "PASS" if error_count == 0 else "FAIL",
    }


def discover_files(raw_root, bronze_root):
    return {
        "events_csv": latest_match(raw_root, "events.csv"),
        "category_tree_csv": latest_match(raw_root, "category_tree.csv"),
        "item_properties_part1_csv": latest_match(raw_root, "item_properties_part1.csv"),
        "item_properties_part2_csv": latest_match(raw_root, "item_properties_part2.csv"),
        "products_raw_json": latest_match(raw_root, "products_raw.json"),
        "categories_raw_json": latest_match(raw_root, "categories_raw.json"),
        "products_parquet": latest_match(bronze_root, "products.parquet"),
        "categories_parquet": latest_match(bronze_root, "categories.parquet"),
    }


def print_discovered_files(files, raw_root, bronze_root):
    print("PROJECT_ROOT:", PROJECT_ROOT)
    print("RAW_ROOT:", raw_root)
    print("BRONZE_ROOT:", bronze_root)
    print("\nDiscovered files:")
    for k, v in files.items():
        print(f"- {k}: {v}")


def validate_file_presence(files, issues, phase):
    required = [
        "events_csv",
        "category_tree_csv",
        "item_properties_part1_csv",
        "item_properties_part2_csv",
        "products_raw_json",
        "categories_raw_json",
    ]
    for key in required:
        if files.get(key) is None:
            add_issue(
                issues,
                dataset=key,
                check_type="file_presence",
                severity="error",
                issue_count=1,
                description=f"Required file not found for {key}. Check PROJECT_ROOT or run ingestion first.",
                phase=phase,
            )


def validate_retailrocket(files, issues, summaries, phase):
    if not all([
        files["events_csv"],
        files["category_tree_csv"],
        files["item_properties_part1_csv"],
        files["item_properties_part2_csv"],
    ]):
        return

    events = pd.read_csv(files["events_csv"])
    category_tree = pd.read_csv(files["category_tree_csv"])
    item_properties_1 = pd.read_csv(files["item_properties_part1_csv"])
    item_properties_2 = pd.read_csv(files["item_properties_part2_csv"])
    item_properties = pd.concat([item_properties_1, item_properties_2], ignore_index=True)

    exp_events = ["timestamp", "visitorid", "event", "itemid", "transactionid"]
    exp_category = ["categoryid", "parentid"]
    exp_item_props = ["timestamp", "itemid", "property", "value"]

    mc = missing_columns(events, exp_events)
    if mc:
        add_issue(issues, "retailrocket_events", "schema", "error", len(mc), f"Missing columns: {mc}", files["events_csv"], phase)
    for col, cnt in null_summary(events).items():
        add_issue(issues, "retailrocket_events", "missing_values", "warning", cnt, f"Nulls in {col}", files["events_csv"], phase)
    dups = duplicate_count(events, subset=["timestamp", "visitorid", "event", "itemid"])
    if dups > 0:
        add_issue(issues, "retailrocket_events", "duplicates", "warning", dups, "Duplicate rows", files["events_csv"], phase)
    bad_event = invalid_membership_count(events, "event", ALLOWED_EVENT_TYPES)
    if bad_event and bad_event > 0:
        add_issue(issues, "retailrocket_events", "domain", "error", bad_event, "Invalid event values", files["events_csv"], phase)
    bad_ts = invalid_timestamp_count(events, "timestamp", unit="ms")
    if bad_ts and bad_ts > 0:
        add_issue(issues, "retailrocket_events", "format", "error", bad_ts, "Invalid timestamps", files["events_csv"], phase)
    summaries.append(build_dataset_summary("retailrocket_events", events, issues, phase))
```

```
    mc = missing_columns(category_tree, exp_category)
    if mc:
        add_issue(issues, "retailrocket_category_tree", "schema", "error", len(mc), f"Missing columns: {mc}", files["category_tree_
    for col, cnt in null_summary(category_tree).items():
        add_issue(issues, "retailrocket_category_tree", "missing_values", "warning", cnt, f"Nulls in {col}", files["category_tree_
    dups = duplicate_count(category_tree, subset=["categoryid", "parentid"])
    if dups > 0:
        add_issue(issues, "retailrocket_category_tree", "duplicates", "warning", dups, "Duplicate rows", files["category_tree_csv"
    summaries.append(build_dataset_summary("retailrocket_category_tree", category_tree, issues, phase))

    mc = missing_columns(item_properties, exp_item_props)
    if mc:
        add_issue(issues, "retailrocket_item_properties", "schema", "error", len(mc), f"Missing columns: {mc}", str(files["item_pr
    for col, cnt in null_summary(item_properties).items():
        add_issue(issues, "retailrocket_item_properties", "missing_values", "warning", cnt, f"Nulls in {col}", str(files["item_pro
    dups = duplicate_count(item_properties, subset=["timestamp", "itemid", "property", "value"])
    if dups > 0:
        add_issue(issues, "retailrocket_item_properties", "duplicates", "warning", dups, "Duplicate rows", str(files["item_propert
    bad_ts = invalid_timestamp_count(item_properties, "timestamp", unit="ms")
    if bad_ts and bad_ts > 0:
        add_issue(issues, "retailrocket_item_properties", "format", "error", bad_ts, "Invalid timestamps", str(files["item_propert
    summaries.append(build_dataset_summary("retailrocket_item_properties", item_properties, issues, phase))


def validate_dummyjson(files, issues, summaries, phase):
    if not all([files["products_raw_json"], files["categories_raw_json"]]):
        return

    with open(files["products_raw_json"], "r", encoding="utf-8") as f:
        products_payload = json.load(f)

    with open(files["categories_raw_json"], "r", encoding="utf-8") as f:
        categories_payload = json.load(f)

    if "products" not in products_payload:
        add_issue(issues, "dummyjson_products_raw", "schema", "error", 1, "Missing 'products' key", files["products_raw_json"], pha
        products_df = pd.DataFrame()
    else:
        products_df = pd.json_normalize(products_payload["products"], sep="_")

    if isinstance(categories_payload, list):
        if len(categories_payload) > 0 and isinstance(categories_payload[0], dict):
            categories_df = pd.json_normalize(categories_payload, sep="_")
        else:
            categories_df = pd.DataFrame({"category": categories_payload})
    else:
        categories_df = pd.DataFrame()
        add_issue(issues, "dummyjson_categories_raw", "schema", "error", 1, "categories_raw.json must be a list", files["categorie

    exp_products = ["id", "title", "category", "price"]

    mc = missing_columns(products_df, exp_products)
    if mc:
        add_issue(issues, "dummyjson_products_raw", "schema", "error", len(mc), f"Missing columns: {mc}", files["products_raw_json
    for col, cnt in null_summary(products_df).items():
        add_issue(issues, "dummyjson_products_raw", "missing_values", "warning", cnt, f"Nulls in {col}", files["products_raw_json"
    dups = duplicate_count(products_df, subset=["id"]) if "id" in products_df.columns else 0
    if dups > 0:
        add_issue(issues, "dummyjson_products_raw", "duplicates", "error", dups, "Duplicate product IDs", files["products_raw_json
    bad_price = invalid_numeric_range_count(products_df, "price", min_value=0)
    if bad_price and bad_price > 0:
        add_issue(issues, "dummyjson_products_raw", "range", "error", bad_price, "Negative prices", files["products_raw_json"], pha
    bad_rating = invalid_numeric_range_count(products_df, "rating", min_value=1, max_value=5)
    if bad_rating and bad_rating > 0:
        add_issue(issues, "dummyjson_products_raw", "range", "warning", bad_rating, "Ratings outside 1-5", files["products_raw_jso
    bad_stock = invalid_numeric_range_count(products_df, "stock", min_value=0)
    if bad_stock and bad_stock > 0:
        add_issue(issues, "dummyjson_products_raw", "range", "error", bad_stock, "Negative stock", files["products_raw_json"], pha
    summaries.append(build_dataset_summary("dummyjson_products_raw", products_df, issues, phase))

    mc = missing_columns(categories_df, ["category"])
    if mc:
        add_issue(issues, "dummyjson_categories_raw", "schema", "error", len(mc), f"Missing columns: {mc}", files["categories_raw_
    for col, cnt in null_summary(categories_df).items():
        add_issue(issues, "dummyjson_categories_raw", "missing_values", "warning", cnt, f"Nulls in {col}", files["categories_raw_j
    dups = duplicate_count(categories_df, subset=["category"]) if "category" in categories_df.columns else 0
    if dups > 0:
        add_issue(issues, "dummyjson_categories_raw", "duplicates", "warning", dups, "Duplicate categories", files["categories_raw_
    summaries.append(build_dataset_summary("dummyjson_categories_raw", categories_df, issues, phase))


def validate_bronze(files, issues, summaries, phase):
    for key in ["products_parquet", "categories_parquet"]:
        path = files.get(key)
        if path is None:
            continue

        df = pd.read_parquet(path)
        required_meta = ["source_system", "source_file", "batch_id", "ingestion_ts"]

        mc = missing_columns(df, required_meta)
        if mc:
            add_issue(issues, key, "schema", "warning", len(mc), f"Missing bronze metadata columns: {mc}", path, phase)
```

```
        if len(df) == 0:
            add_issue(issues, key, "completeness", "error", 1, "Empty bronze file", path, phase)

        summaries.append(build_dataset_summary(key, df, issues, phase))


def run_validation_cycle(files, phase):
    issues = []
    summaries = []

    validate_file_presence(files, issues, phase)
    validate_retailrocket(files, issues, summaries, phase)
    validate_dummyjson(files, issues, summaries, phase)
    validate_bronze(files, issues, summaries, phase)

    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    if issues_df.empty:
        issues_df = pd.DataFrame(columns=["phase", "dataset", "check_type", "severity", "issue_count", "description", "file_path"]
    if summary_df.empty:
        summary_df = pd.DataFrame(columns=["phase", "dataset", "rows", "columns", "errors", "warnings", "status"])

    overall_status = "PASS" if len(issues_df[issues_df["severity"] == "error"]) == 0 else "FAIL"

    return issues_df, summary_df, overall_status


def ensure_columns(df, columns):
    for col in columns:
        if col not in df.columns:
            df[col] = pd.NA
    return df


def save_json(path, payload):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, ensure_ascii=False)


def fix_events(src_path, dst_path, fixes):
    df = pd.read_csv(src_path)
    rows_before = len(df)

    expected = ["timestamp", "visitorid", "event", "itemid", "transactionid"]
    missing = [c for c in expected if c not in df.columns]
    df = ensure_columns(df, expected)
    if missing:
        add_fix(fixes, "retailrocket_events", "add_missing_columns", rows_before, len(df), f"Added columns: {missing}", src_path)

    if "event" in df.columns:
        df["event"] = df["event"].astype(str).str.strip().str.lower()
        before = len(df)
        df = df[df["event"].isin(ALLOWED_EVENT_TYPES) | df["event"].isna()]
        add_fix(fixes, "retailrocket_events", "remove_invalid_event_values", before, len(df), "Kept only allowed event types", src

    if "timestamp" in df.columns:
        df["timestamp"] = pd.to_numeric(df["timestamp"], errors="coerce")
        before = len(df)
        df = df[df["timestamp"].notna()]
        add_fix(fixes, "retailrocket_events", "drop_invalid_timestamps", before, len(df), "Removed rows with invalid timestamp", s

    for col in ["visitorid", "itemid", "transactionid"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    before = len(df)
    df = df.drop_duplicates(subset=["timestamp", "visitorid", "event", "itemid"])
    add_fix(fixes, "retailrocket_events", "drop_duplicates", before, len(df), "Removed duplicate event rows", src_path)

    before = len(df)
    df = df.dropna(subset=["timestamp", "visitorid", "event", "itemid"])
    add_fix(fixes, "retailrocket_events", "drop_missing_required_rows", before, len(df), "Dropped rows missing required fields", s

    df.to_csv(dst_path, index=False)
    return dst_path


def fix_category_tree(src_path, dst_path, fixes):
    df = pd.read_csv(src_path)
    rows_before = len(df)

    expected = ["categoryid", "parentid"]
    missing = [c for c in expected if c not in df.columns]
    df = ensure_columns(df, expected)
    if missing:
        add_fix(fixes, "retailrocket_category_tree", "add_missing_columns", rows_before, len(df), f"Added columns: {missing}", src

    for col in ["categoryid", "parentid"]:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    before = len(df)
    df = df.dropna(subset=["categoryid"])
```

```
    add_fix(fixes, "retailrocket_category_tree", "drop_missing_categoryid", before, len(df), "Dropped rows with missing categoryid

    before = len(df)
    df = df.drop_duplicates(subset=["categoryid", "parentid"])
    add_fix(fixes, "retailrocket_category_tree", "drop_duplicates", before, len(df), "Removed duplicate category rows", src_path)

    df.to_csv(dst_path, index=False)
    return dst_path


def fix_item_properties(part1_path, part2_path, dst_part1_path, dst_part2_path, fixes):
    df1 = pd.read_csv(part1_path)
    df2 = pd.read_csv(part2_path)
    df = pd.concat([df1, df2], ignore_index=True)
    rows_before = len(df)

    expected = ["timestamp", "itemid", "property", "value"]
    missing = [c for c in expected if c not in df.columns]
    df = ensure_columns(df, expected)
    if missing:
        add_fix(fixes, "retailrocket_item_properties", "add_missing_columns", rows_before, len(df), f"Added columns: {missing}", pa

    df["timestamp"] = pd.to_numeric(df["timestamp"], errors="coerce")
    df["itemid"] = pd.to_numeric(df["itemid"], errors="coerce")
    df["property"] = df["property"].astype("string").str.strip()
    df["value"] = df["value"].astype("string").str.strip()

    before = len(df)
    df = df.dropna(subset=["timestamp", "itemid", "property", "value"])
    add_fix(fixes, "retailrocket_item_properties", "drop_missing_required_rows", before, len(df), "Dropped rows with invalid requi

    before = len(df)
    df = df.drop_duplicates(subset=["timestamp", "itemid", "property", "value"])
    add_fix(fixes, "retailrocket_item_properties", "drop_duplicates", before, len(df), "Removed duplicate property rows", part1_pa

    split_index = len(df) // 2
    df.iloc[:split_index].to_csv(dst_part1_path, index=False)


[TRUNCATED FOR PDF SIZE]
```

Code: src\02-validation\run_validation.txt


```
{
 "cells": [
  {
   "cell_type": "code",
   "execution_count": 2,
   "id": "fef32579-f890-4b53-8d24-2fcae64a4914",
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "PROJECT_ROOT: C:\\Users\\barath\\recomart-pipeline\n",
      "RAW_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\raw\n",
      "BRONZE_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\n",
      "\n",
      "Discovered files:\n",
      "- events_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour=14\\events.csv
      "- category_tree_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrocket\\load_date=2026-04-26\\load_hour=19\\cat
      "- item_properties_part1_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour
      "- item_properties_part2_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour
      "- products_raw_json: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\produc
      "- categories_raw_json: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\cate
      "- products_parquet: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\\dummyjson\\products.parquet\n",
      "- categories_parquet: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\\dummyjson\\categories.parquet\n",
      "\n",
      "Initial status: FAIL\n",
      "Final status: PASS\n",
      "Initial issues file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\validation_issues_initial_20260428T144031Z.
      "Initial summary file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\validation_summary_initial_20260428T144031
      "Revalidation issues file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\validation_issues_revalidated_20260428
      "Revalidation summary file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\validation_summary_revalidated_202604
      "Fix log file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\fix_log_20260428T144031Z.csv\n",
      "JSON report file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.json\n",
      "PDF report file: C:\\Users\\barath\\recomart-pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.pdf\n"
     ]
    }
   ],
   "source": [
    "# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report\n",
    "\n",
    "import json\n",
    "from pathlib import Path\n",
    "from datetime import datetime, timezone\n",
    "\n",
    "import pandas as pd\n",
    "\n",
```

```
"from reportlab.lib import colors\n",
"from reportlab.lib.pagesizes import A4\n",
"from reportlab.lib.styles import getSampleStyleSheet\n",
"from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, PageBreak\n",
"\n",
"# ============================================================\n",
"# RecoMart Validation - robust file discovery + validation\n",
"# with remediation, revalidation, and PDF reporting\n",
"# ============================================================\n",
"\n",
"PROJECT_ROOT = Path.cwd().resolve()\n",
"while not (PROJECT_ROOT / \"src\").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:\n",
"    PROJECT_ROOT = PROJECT_ROOT.parent\n",
"\n",
"RAW_ROOT = PROJECT_ROOT / \"data\" / \"raw\"\n",
"BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"bronze\"\n",
"\n",
"REMEDIATED_RAW_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"raw\"\n",
"REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"bronze\"\n",
"\n",
"REPORT_ROOT = PROJECT_ROOT / \"reports\" / \"validation\"\n",
"LOG_ROOT = PROJECT_ROOT / \"logs\"\n",
"\n",
"for folder in [REPORT_ROOT, LOG_ROOT, REMEDIATED_RAW_ROOT, REMEDIATED_BRONZE_ROOT]:\n",
"    folder.mkdir(parents=True, exist_ok=True)\n",
"\n",
"UTC_NOW = datetime.now(timezone.utc)\n",
"RUN_ID = UTC_NOW.strftime(\"%Y%m%dT%H%M%SZ\")\n",
"RUN_TS = UTC_NOW.isoformat()\n",
"\n",
"LOG_FILE = LOG_ROOT / f\"validation_log_{RUN_ID}.jsonl\"\n",
"INITIAL_ISSUES_FILE = REPORT_ROOT / f\"validation_issues_initial_{RUN_ID}.csv\"\n",
"INITIAL_SUMMARY_FILE = REPORT_ROOT / f\"validation_summary_initial_{RUN_ID}.csv\"\n",
"FINAL_ISSUES_FILE = REPORT_ROOT / f\"validation_issues_revalidated_{RUN_ID}.csv\"\n",
"FINAL_SUMMARY_FILE = REPORT_ROOT / f\"validation_summary_revalidated_{RUN_ID}.csv\"\n",
"FIX_LOG_FILE = REPORT_ROOT / f\"fix_log_{RUN_ID}.csv\"\n",
"REPORT_FILE = REPORT_ROOT / f\"data_quality_report_{RUN_ID}.json\"\n",
"PDF_REPORT_FILE = REPORT_ROOT / f\"data_quality_report_{RUN_ID}.pdf\"\n",
"\n",
"ALLOWED_EVENT_TYPES = {\"view\", \"addtocart\", \"transaction\"}\n",
"\n",
"\n",
"def log_event(stage, status, message, extra=None):\n",
"    record = {\n",
"        \"event_ts\": datetime.now(timezone.utc).isoformat(),\n",
"        \"stage\": stage,\n",
"        \"status\": status,\n",
"        \"message\": message,\n",
"        \"run_id\": RUN_ID,\n",
"    }\n",
"    if extra:\n",
"        record.update(extra)\n",
"    with open(LOG_FILE, \"a\", encoding=\"utf-8\") as f:\n",
"        f.write(json.dumps(record) + \"\\n\")\n",
"\n",
"\n",
"def add_issue(issues, dataset, check_type, severity, issue_count, description, file_path=None, phase=\"initial\"):\n",
"    issues.append({\n",
"        \"phase\": phase,\n",
"        \"dataset\": dataset,\n",
"        \"check_type\": check_type,\n",
"        \"severity\": severity,\n",
"        \"issue_count\": int(issue_count),\n",
"        \"description\": description,\n",
"        \"file_path\": str(file_path) if file_path else None,\n",
"    })\n",
"\n",
"\n",
"def add_fix(fixes, dataset, action, rows_before=None, rows_after=None, details=None, file_path=None):\n",
"    fixes.append({\n",
"        \"dataset\": dataset,\n",
"        \"action\": action,\n",
"        \"rows_before\": rows_before,\n",
"        \"rows_after\": rows_after,\n",
"        \"details\": details,\n",
"        \"file_path\": str(file_path) if file_path else None,\n",
"    })\n",
"\n",
"\n",
"def latest_match(base_dir: Path, pattern: str):\n",
"    if not base_dir.exists():\n",
"        return None\n",
"    matches = list(base_dir.rglob(pattern))\n",
"    if not matches:\n",
"        return None\n",
"    return max(matches, key=lambda p: p.stat().st_mtime)\n",
"\n",
"\n",
"def missing_columns(df, expected_columns):\n",
"    return [c for c in expected_columns if c not in df.columns]\n",
"\n",
"\n",
"def duplicate_count(df, subset=None):\n",
```

```
"    return int(df.duplicated(subset=subset).sum())\n",
"\n",
"\n",
"def null_summary(df):\n",
"    result = {}\n",
"    for c in df.columns:\n",
"        n = int(df[c].isna().sum())\n",
"        if n > 0:\n",
"            result[c] = n\n",
"    return result\n",
"\n",
"\n",
"def invalid_numeric_range_count(df, column, min_value=None, max_value=None):\n",
"    if column not in df.columns:\n",
"        return None\n",
"    s = pd.to_numeric(df[column], errors=\"coerce\")\n",
"    mask = pd.Series(False, index=df.index)\n",
"    if min_value is not None:\n",
"        mask = mask | (s < min_value)\n",
"    if max_value is not None:\n",
"        mask = mask | (s > max_value)\n",
"    return int(mask.fillna(False).sum())\n",
"\n",
"\n",
"def invalid_membership_count(df, column, allowed_values):\n",
"    if column not in df.columns:\n",
"        return None\n",
"    return int((~df[column].isin(allowed_values)).sum())\n",
"\n",
"\n",
"def invalid_timestamp_count(df, column, unit=None):\n",
"    if column not in df.columns:\n",
"        return None\n",
"    parsed = pd.to_datetime(df[column], errors=\"coerce\", unit=unit)\n",
"    return int(parsed.isna().sum())\n",
"\n",
"\n",
"def build_dataset_summary(dataset_name, df, issues, phase):\n",
"    dataset_issues = [x for x in issues if x[\"dataset\"] == dataset_name and x[\"phase\"] == phase]\n",
"    error_count = sum(x[\"issue_count\"] for x in dataset_issues if x[\"severity\"] == \"error\")\n",
"    warning_count = sum(x[\"issue_count\"] for x in dataset_issues if x[\"severity\"] == \"warning\")\n",
"    return {\n",
"        \"phase\": phase,\n",
"        \"dataset\": dataset_name,\n",
"        \"rows\": int(len(df)) if df is not None else 0,\n",
"        \"columns\": int(len(df.columns)) if df is not None else 0,\n",
"        \"errors\": int(error_count),\n",
"        \"warnings\": int(warning_count),\n",
"        \"status\": \"PASS\" if error_count == 0 else \"FAIL\",\n",
"    }\n",
"\n",
"\n",
"def discover_files(raw_root, bronze_root):\n",
"    return {\n",
"        \"events_csv\": latest_match(raw_root, \"events.csv\"),\n",
"        \"category_tree_csv\": latest_match(raw_root, \"category_tree.csv\"),\n",
"        \"item_properties_part1_csv\": latest_match(raw_root, \"item_properties_part1.csv\"),\n",
"        \"item_properties_part2_csv\": latest_match(raw_root, \"item_properties_part2.csv\"),\n",
"        \"products_raw_json\": latest_match(raw_root, \"products_raw.json\"),\n",
"        \"categories_raw_json\": latest_match(raw_root, \"categories_raw.json\"),\n",
"        \"products_parquet\": latest_match(bronze_root, \"products.parquet\"),\n",
"        \"categories_parquet\": latest_match(bronze_root, \"categories.parquet\"),\n",
"    }\n",
"\n",
"\n",
"def print_discovered_files(files, raw_root, bronze_root):\n",
"    print(\"PROJECT_ROOT:\", PROJECT_ROOT)\n",
"    print(\"RAW_ROOT:\", raw_root)\n",
"    print(\"BRONZE_ROOT:\", bronze_root)\n",
"    print(\"\\nDiscovered files:\")\n",
"    for k, v in files.items():\n",
"        print(f\"- {k}: {v}\")\n",
"\n",
"\n",
"def validate_file_presence(files, issues, phase):\n",
"    required = [\n",
"        \"events_csv\",\n",
"        \"category_tree_csv\",\n",
"        \"item_properties_part1_csv\",\n",
"        \"item_properties_part2_csv\",\n",
"        \"products_raw_json\",\n",
"        \"categories_raw_json\",\n",
"    ]\n",
"    for key in required:\n",
"        if files.get(key) is None:\n",
"            add_issue(\n",
"                issues,\n",
"                dataset=key,\n",
"                check_type=\"file_presence\",\n",
"                severity=\"error\",\n",
"                issue_count=1,\n",
"                description=f\"Required file not found for {key}. Check PROJECT_ROOT or run ingestion first.\",\n",
"                phase=phase,\n",
```

```
"            )\n",
"\n",
"\n",
"def validate_retailrocket(files, issues, summaries, phase):\n",
"    if not all([\n",
"        files[\"events_csv\"],\n",
"        files[\"category_tree_csv\"],\n",
"        files[\"item_properties_part1_csv\"],\n",
"        files[\"item_properties_part2_csv\"],\n",
"    ]):\n",
"        return\n",
"\n",
"    events = pd.read_csv(files[\"events_csv\"])\n",
"    category_tree = pd.read_csv(files[\"category_tree_csv\"])\n",
"    item_properties_1 = pd.read_csv(files[\"item_properties_part1_csv\"])\n",
"    item_properties_2 = pd.read_csv(files[\"item_properties_part2_csv\"])\n",
"    item_properties = pd.concat([item_properties_1, item_properties_2], ignore_index=True)\n",
"\n",
"    exp_events = [\"timestamp\", \"visitorid\", \"event\", \"itemid\", \"transactionid\"]\n",
"    exp_category = [\"categoryid\", \"parentid\"]\n",
"    exp_item_props = [\"timestamp\", \"itemid\", \"property\", \"value\"]\n",
"\n",
"    mc = missing_columns(events, exp_events)\n",
"    if mc:\n",
"        add_issue(issues, \"retailrocket_events\", \"schema\", \"error\", len(mc), f\"Missing columns: {mc}\", files[\"events_
"    for col, cnt in null_summary(events).items():\n",
"        add_issue(issues, \"retailrocket_events\", \"missing_values\", \"warning\", cnt, f\"Nulls in {col}\", files[\"events_c
"    dups = duplicate_count(events, subset=[\"timestamp\", \"visitorid\", \"event\", \"itemid\"])\n",
"    if dups > 0:\n",
"        add_issue(issues, \"retailrocket_events\", \"duplicates\", \"warning\", dups, \"Duplicate rows\", files[\"events_csv\
"    bad_event = invalid_membership_count(events, \"event\", ALLOWED_EVENT_TYPES)\n",
"    if bad_event and bad_event > 0:\n",
"        add_issue(issues, \"retailrocket_events\", \"domain\", \"error\", bad_event, \"Invalid event values\", files[\"events_
"    bad_ts = invalid_timestamp_count(events, \"timestamp\", unit=\"ms\")\n",
"    if bad_ts and bad_ts > 0:\n",
"        add_issue(issues, \"retailrocket_events\", \"format\", \"error\", bad_ts, \"Invalid timestamps\", files[\"events_csv\
"    summaries.append(build_dataset_summary(\"retailrocket_events\", events, issues, phase))\n",
"\n",
"    mc = missing_columns(category_tree, exp_category)\n",
"    if mc:\n",
"        add_issue(issues, \"retailrocket_category_tree\", \"schema\", \"error\", len(mc), f\"Missing columns: {mc}\", files[\
"    for col, cnt in null_summary(category_tree).items():\n",
"        add_issue(issues, \"retailrocket_category_tree\", \"missing_values\", \"warning\", cnt, f\"Nulls in {col}\", files[\"
"    dups = duplicate_count(category_tree, subset=[\"categoryid\", \"parentid\"])\n",
"    if dups > 0:\n",
"        add_issue(issues, \"retailrocket_category_tree\", \"duplicates\", \"warning\", dups, \"Duplicate rows\", files[\"cate
"    summaries.append(build_dataset_summary(\"retailrocket_category_tree\", category_tree, issues, phase))\n",
"\n",
"    mc = missing_columns(item_properties, exp_item_props)\n",
"    if mc:\n",
"        add_issue(issues, \"retailrocket_item_properties\", \"schema\", \"error\", len(mc), f\"Missing columns: {mc}\", str(f
"    for col, cnt in null_summary(item_properties).items():\n",
"        add_issue(issues, \"retailrocket_item_properties\", \"missing_values\", \"warning\", cnt, f\"Nulls in {col}\", str(fi
"    dups = duplicate_count(item_properties, subset=[\"timestamp\", \"itemid\", \"property\", \"value\"])\n",
"    if dups > 0:\n",
"        add_issue(issues, \"retailrocket_item_properties\", \"duplicates\", \"warning\", dups, \"Duplicate rows\", str(files[
"    bad_ts = invalid_timestamp_count(item_properties, \"timestamp\", unit=\"ms\")\n",
"    if bad_ts and bad_ts > 0:\n",
"        add_issue(issues, \"retailrocket_item_properties\", \"format\", \"error\", bad_ts, \"Invalid timestamps\", str(files[
"    summaries.append(build_dataset_summary(\"retailrocket_item_properties\", item_properties, issues, phase))\n",
"\n",
"\n",
"def validate_dummyjson(files, issues, summaries, phase):\n",
"    if not all([files[\"products_raw_json\"], files[\"categories_raw_json\"]]):\n",
"        return\n",
"\n",
"    with open(files[\"products_raw_json\"], \"r\", encoding=\"utf-8\") as f:\n",
"        products_payload = json.load(f)\n",
"\n",
"    with open(files[\"categories_raw_json\"], \"r\", encoding=\"utf-8\") as f:\n",
"        categories_payload = json.load(f)\n",
"\n",
"    if \"products\" not in products_payload:\n",
"        add_issue(issues, \"dummyjson_products_raw\", \"schema\", \"error\", 1, \"Missing 'products' key\", files[\"products_
"        products_df = pd.DataFrame()\n",
"    else:\n",
"        products_df = pd.json_normalize(products_payload[\"products\"], sep=\"_\")\n",
"\n",
"    if isinstance(categories_payload, list):\n",
"        if len(categories_payload) > 0 and isinstance(categories_payload[0], dict):\n",
"            categories_df = pd.json_normalize(categories_payload, sep=\"_\")\n",
"        else:\n",
"            categories_df = pd.DataFrame({\"category\": categories_payload})\n",
"    else:\n",
"        categories_df = pd.DataFrame()\n",
"        add_issue(issues, \"dummyjson_categories_raw\", \"schema\", \"error\", 1, \"categories_raw.json must be a list\", fil
"\n",
"    exp_products = [\"id\", \"title\", \"category\", \"price\"]\n",
"\n",
"    mc = missing_columns(products_df, exp_products)\n",
"    if mc:\n",
"        add_issue(issues, \"dummyjson_products_raw\", \"schema\", \"error\", len(mc), f\"Missing columns: {mc}\", files[\"pro
"    for col, cnt in null_summary(products_df).items():\n",
```

```
    "        add_issue(issues, \"dummyjson_products_raw\", \"missing_values\", \"warning\", cnt, f\"Nulls in {col}\", files[\"prod
    "    dups = duplicate_count(products_df, subset=[\"id\"]) if \"id\" in products_df.columns else 0\n",
    "    if dups > 0:\n",
    "        add_issue(issues, \"dummyjson_products_raw\", \"duplicates\", \"error\", dups, \"Duplicate product IDs\", files[\"pro
    "    bad_price = invalid_numeric_range_count(products_df, \"price\", min_value=0)\n",
    "    if bad_price and bad_price > 0:\n",
    "        add_issue(issues, \"dummyjson_products_raw\", \"range\", \"error\", bad_price, \"Negative prices\", files[\"products_
    "    bad_rating = invalid_numeric_range_count(products_df, \"rating\", min_value=1, max_value=5)\n",
    "    if bad_rating and bad_rating > 0:\n",
    "        add_issue(issues, \"dummyjson_products_raw\", \"range\", \"warning\", bad_rating, \"Ratings outside 1-5\", files[\"pr
    "    bad_stock = invalid_numeric_range_count(products_df, \"stock\", min_value=0)\n",
    "    if bad_stock and bad_stock > 0:\n",
    "        add_issue(issues, \"dummyjson_products_raw\", \"range\", \"error\", bad_stock, \"Negative stock\", files[\"products_ra
    "    summaries.append(build_dataset_summary(\"dummyjson_products_raw\", products_df, issues, phase))\n",
    "\n",
    "    mc = missing_columns(categories_df, [\"category\"])\n",
    "    if mc:\n",
    "        add_issue(issues, \"dummyjson_categories_raw\", \"schema\", \"error\", len(mc), f\"Missing columns: {mc}\", files[\"ca
    "    for col, cnt in null_summary(categories_df).items():\n",
    "        add_issue(issues, \"dummyjson_categories_raw\", \"missing_values\", \"warning\", cnt, f\"Nulls in {col}\", files[\"ca
    "    dups = duplicate_count(categories_df, subset=[\"category\"]) if \"category\" in categories_df.columns else 0\n",
    "    if dups > 0:\n",
    "        add_issue(issues, \"dummyjson_categories_raw\", \"duplicates\", \"warning\", dups, \"Duplicate categories\", files[\"
    "
```

[TRUNCATED FOR PDF SIZE]

Code: src\03-eda-prep\eda_prep copy 2.txt

```
import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)


# ----------------------------
# Project paths and setup
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)


# ----------------------------
# File discovery helpers
# ----------------------------
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
```

```
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)


files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")


# ----------------------------
# Validation helpers
# ----------------------------
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }


def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")
    return {
        "dataset": dataset,
        "rows": int(len(df)),
        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }


def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)


def profile_df(df, name):
    summary = pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())


def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)

    print(f"Saved: {issues_path}")
    print(f"Saved: {summary_path}")

    return issues_df, summary_df, issues_path, summary_path


def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")
```

```
    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback


# ----------------------------
# Raw validation functions
# ----------------------------
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["timestamp", "visitorid", "event", "itemid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    if "transactionid" not in d.columns:
        d["transactionid"] = np.nan

    item = make_issue(
        "retailrocket_events",
        "missing_values",
        "warning",
        d["transactionid"].isna().sum(),
        "Nulls in transactionid",
        file_path
    )
    if item:
        issues.append(item)

    item = make_issue(
        "retailrocket_events",
        "duplicates",
        "warning",
        d.duplicated().sum(),
        "Duplicate rows",
        file_path
    )
    if item:
        issues.append(item)

    return issues


def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file
    if item:
        issues.append(item)
        return issues

    item = make_issue(
        "retailrocket_category_tree",
        "missing_values",
        "warning",
        d["parentid"].isna().sum(),
        "Nulls in parentid",
        file_path
    )
    if item:
        issues.append(item)

    return issues


def validate_raw_products(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["id", "title", "category", "price"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("dummyjson_products_raw", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_pat
    if item:
        issues.append(item)

    if "brand" in d.columns:
        item = make_issue(
```

```
            "dummyjson_products_raw",
            "missing_values",
            "warning",
            d["brand"].isna().sum(),
            "Nulls in brand",
            file_path
        )
        if item:
            issues.append(item)

    return issues


def validate_raw_categories(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    has_valid_category_field = any(col in d.columns for col in ["category", "slug", "name"])
    item = make_issue(
        "dummyjson_categories_raw",
        "schema",
        "error",
        0 if has_valid_category_field else 1,
        "Expected one of ['category', 'slug', 'name'] in categories payload",
        file_path
    )
    if item:
        issues.append(item)

    return issues


# ----------------------------
# Prepared validation functions
# ----------------------------
def validate_prepared_interactions(df, file_path):
    issues = []
    required = ["user_id", "item_id", "event", "event_weight", "event_ts"]
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("interactions_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path
    if item:
        issues.append(item)
        return issues

    item = make_issue("interactions_prepared", "duplicates", "warning", df.duplicated().sum(), "Duplicate rows", file_path)
    if item:
        issues.append(item)

    bad_txn = ((df["event"] == "transaction") & (df["transactionid"].isna())).sum()
    item = make_issue("interactions_prepared", "missing_values", "error", bad_txn, "Null transactionid for transaction events", fi
    if item:
        issues.append(item)

    return issues


def validate_prepared_products(df, file_path):
    issues = []
    required = ["product_id", "title", "category", "price"]
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("products_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    item = make_issue("products_prepared", "duplicates", "warning", df.duplicated().sum(), "Duplicate rows", file_path)
    if item:
        issues.append(item)

    return issues


def validate_prepared_categories(df, file_path):
    issues = []
    required = ["category"]
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("categories_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    blanks = (df["category"].astype(str).str.strip() == "").sum()
    item = make_issue("categories_prepared", "missing_values", "error", blanks, "Blank category values", file_path)
    if item:
        issues.append(item)

    return issues


# ----------------------------
# Load the raw ingestion files
# ----------------------------
```

```
events_raw = pd.read_csv(files["events_csv"])
category_tree_raw = pd.read_csv(files["category_tree_csv"])
item_props_1 = pd.read_csv(files["item_properties_part1_csv"])
item_props_2 = pd.read_csv(files["item_properties_part2_csv"])
item_properties_raw = pd.concat([item_props_1, item_props_2], ignore_index=True)

with open(files["products_raw_json"], "r", encoding="utf-8") as f:
    products_payload = json.load(f)

with open(files["categories_raw_json"], "r", encoding="utf-8") as f:
    categories_payload = json.load(f)

products_raw = pd.json_normalize(products_payload.get("products", []), sep="_")

if isinstance(categories_payload, list):
    if len(categories_payload) > 0 and isinstance(categories_payload[0], dict):
        categories_raw = pd.json_normalize(categories_payload, sep="_")
    else:
        categories_raw = pd.DataFrame({"category": categories_payload})
elif isinstance(categories_payload, dict):
    categories_raw = pd.json_normalize(categories_payload, sep="_")
else:
    categories_raw = pd.DataFrame()

print("events_raw:", events_raw.shape)
print("category_tree_raw:", category_tree_raw.shape)
print("item_properties_raw:", item_properties_raw.shape)
print("products_raw:", products_raw.shape)
print("categories_raw:", categories_raw.shape)


# ----------------------------
# Quick profiling snapshot
# ----------------------------
profile_df(events_raw, "events_raw")
profile_df(category_tree_raw, "category_tree_raw")
profile_df(item_properties_raw, "item_properties_raw")
profile_df(products_raw, "products_raw")
profile_df(categories_raw, "categories_raw")


# ----------------------------
# Raw validation
# ----------------------------
raw_issues = []
raw_issues.extend(validate_raw_events(events_raw, files["events_csv"]))
raw_issues.extend(validate_raw_category_tree(category_tree_raw, files["category_tree_csv"]))
raw_issues.extend(validate_raw_products(products_raw, files["products_raw_json"]))
raw_issues.extend(validate_raw_categories(categories_raw, files["categories_raw_json"]))

raw_summaries = [
    summarize_validation("retailrocket_events", events_raw, [i for i in raw_issues if i["dataset"] == "retailrocket_events"]),
    summarize_validation("retailrocket_category_tree", category_tree_raw, [i for i in raw_issues if i["dataset"] == "retailrocket_
    summarize_validation("dummyjson_products_raw", products_raw, [i for i in raw_issues if i["dataset"] == "dummyjson_products_raw
    summarize_validation("dummyjson_categories_raw", categories_raw, [i for i in raw_issues if i["dataset"] == "dummyjson_categori
]

raw_issues_df, raw_summary_df, raw_issues_path, raw_summary_path = save_validation_reports(raw_issues, raw_summaries, "raw_validat

print("\nRAW VALIDATION ISSUES")
display(raw_issues_df)

print("\nRAW VALIDATION SUMMARY")
display(raw_summary_df)


# ----------------------------
# Clean and prepare Retailrocket interactions
# ----------------------------
events = events_raw.copy()

events.columns = [c.strip().lower() for c in events.columns]

required_event_cols = ["timestamp", "visitorid", "event", "itemid"]
missing_cols = [c for c in required_event_cols if c not in events.columns]
if missing_cols:
    raise ValueError(f"events.csv missing columns: {missing_cols}")

events = events.dropna(subset=["timestamp", "visitorid", "event", "itemid"]).copy()

events["timestamp"] = pd.to_numeric(events["timestamp"], errors="coerce")
events["visitorid"] = pd.to_numeric(events["visitorid"], errors="coerce")
events["itemid"] = pd.to_numeric(events["itemid"], errors="coerce")

events = events.dropna(subset=["timestamp", "visitorid", "itemid"]).copy()

events["event"] = events["event"].astype(str).str.strip().str.lower()
events = events[events["event"].isin(["view", "addtocart", "transaction"])].copy()

if "transactionid" not in events.columns:
    events["transactionid"] = np.nan

events["transactionid"] = pd.to_numeric(events["transactionid"], errors="coerce")
```

```
events["transactionid_missing"] = events["transactionid"].isna().astype(int)

bad_transactions = events[(events["event"] == "transaction") & (events["transactionid"].isna())].copy()
if not bad_transactions.empty:
    bad_transactions.to_csv(QUARANTINE_ROOT / f"invalid_transaction_rows_{RUN_TS}.csv", index=False)

events = events[~((events["event"] == "transaction") & (events["transactionid"].isna()))].copy()

events["event_ts"] = pd.to_datetime(events["timestamp"], unit="ms", errors="coerce")
events = events.dropna(subset=["event_ts"]).copy()

events["user_id"] = events["visitorid"].astype("int64")
events["item_id"] = events["itemid"].astype("int64")
events["event_date"] = events["event_ts"].dt.date
events["event_hour"] = events["event_ts"].dt.hour
events["event_dow"] = events["event_ts"].dt.day_name()

event_weight_map = {"view": 1.0, "addtocart": 3.0, "transaction": 5.0}
events["event_weight"] = events["event"].map(event_weight_map).astype(float)

events = events.drop_duplicates(subset=["timestamp", "user_id", "item_id", "event"]).copy()

interactions_prepared = events[
    [
        "user_id",
        "item_id",
        "event",
        "event_weight",
        "event_ts",
        "event_date",
        "event_hour",
        "event_dow",
        "transactionid",
        "transactionid_missing",
    ]
].reset_index(drop=True)

print(interactions_prepared.shape)
display(interactions_prepared.head())


# ----------------------------
# Prepare Retailrocket item attributes snapshot
# ----------------------------
category_tree = category_tree_raw.copy()
category_tree.columns = [c.strip().lower() for c in category_tree.columns]

if "parentid" in category_tree.columns:
    category_tree["is_root"] = category_tree["parentid"].isna().astype(int)
    category_tree["parentid"] = category_tree["parentid"].fillna(-1)

item_properties = item_properties_raw.copy()
item_properties.columns = [c.strip().lower() for c in item_properties.columns]

item_properties = item_properties.dropna(subset=["timestamp", "itemid", "property", "value"]).copy()
item_properties["timestamp"] = pd.to_numeric(item_properties["timestamp"], errors="coerce")
item_properties["itemid"] = pd.to_numeric(item_properties["itemid"], errors="coerce")
item_properties["prop_ts"] = pd.to_datetime(item_properties["timestamp"], unit="ms", errors="coerce")
item_properties = item_properties.dropna(subset=["timestamp", "itemid", "prop_ts"]).copy()

item_properties["item_id"] = item_properties["itemid"].astype("int64")
item_properties["property"] = item_properties["property"].astype(str).str.strip().str.lower()
item_properties["value"] = item_properties["value"].astype(str).str.strip()

latest_item_props = (
    item_properties
    .sort_values(["item_id", "property", "prop_ts"])
    .drop_duplicates(subset=["item_id", "property"], keep="last")
)

top_properties = (
    latest_item_props["property"]
    .value_counts()
    .head(25)
    .index
    .tolist()
)

item_snapshot = (
    latest_item_props[latest_item_props["property"].isin(top_properties)]
    .pivot(index="item_id", columns="property", values="value")
    .reset_index()
)

print("Top properties used:", len(top_properties))
print("item_snapshot:", item_snapshot.shape)
display(item_snapshot.head())


# ----------------------------
# Clean, encode, and normalize DummyJSON products
# ----------------------------
def minmax_scale(series):
```

```
    s = pd.to_numeric(series, errors="coerce")
    s_min = s.min()
    s_max = s.max()
    if pd.isna(s_min) or pd.isna(s_max) or s_min == s_max:
        return pd.Series(np.zeros(len(s)), index=s.index)
    return (s - s_min) / (s_max - s_min)


products = products_raw.copy()
products.columns = [c.strip().lower() for c in products.columns]

required_product_cols = ["id", "title", "category", "price"]
missing_product_cols = [c for c in required_product_cols if c not in products.columns]
if missing_product_cols:
    raise ValueError(f"products_raw.json missing fields: {missing_product_cols}")

products = products.drop_duplicates(subset=["id"]).copy()
products["id"] = pd.to_numeric(products["id"], errors="coerce")
products["price"] = pd.to_numeric(products.get("price"), errors="coerce")
products["rating"] = pd.to_numeric(products.get("rating"), errors="coerce")
products["stock"] = pd.to_numeric(products.get("stock"), errors="coerce")
products["discountpercentage"] = pd.to_numeric(products.get("discountpercentage"), errors="coerce")

products["title"] = products["title"].astype(str).str.strip()
products["category"] = products["category"].astype(str).str.strip().str.lower()

if "brand" not in products.columns:
    products["brand"] = "Unknown"
else:
    products["brand"] = products["brand"].fillna("Unknown").astype(str).str.strip()
    products.loc[products["brand"].eq(""), "brand"] = "Unknown"

products = products.dropna(subset=["id", "title", "category", "price"]).copy()
products = products[products["price"] >= 0].copy()

products["product_id"] = products["id"].astype("int64")
products["price_norm"] = minmax_scale(products["price"])
products["rating_norm"] = minmax_scale(products["rating"])
products["stock_norm"] = minmax_scale(products["stock"])
products["discount_norm"] = minmax_scale(products["discountpercentage"])

category_dummies = pd.get_dummies(products["category"], prefix="cat", dtype=int)
brand_dummies = pd.get_dummies(products["brand"], prefix="brand", dtype=int)

products_prepared = pd.concat(
    [
        products[
            [
                "product_id",
                "title",
                "category",
                "brand",
                "price",
                "rating",
                "stock",
                "discountpercentage",
                "price

[TRUNCATED FOR PDF SIZE]
```

Code: src\03-eda-prep\eda_prep-copy.ipynb

```
import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)


# ----------------------------
# Project paths and setup
# ----------------------------
```

```
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)


# ----------------------------
# File discovery helpers
# ----------------------------
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)


files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")


# ----------------------------
# Validation helpers
# ----------------------------
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }


def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")
    return {
        "dataset": dataset,
        "rows": int(len(df)),
        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }


def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)


def profile_df(df, name):
    summary = pd.DataFrame({
```

```
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())


def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)

    print(f"Saved: {issues_path}")
    print(f"Saved: {summary_path}")

    return issues_df, summary_df, issues_path, summary_path


def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")

    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback


# ----------------------------
# Raw validation functions
# ----------------------------
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["timestamp", "visitorid", "event", "itemid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    if "transactionid" not in d.columns:
        d["transactionid"] = np.nan

    item = make_issue(
        "retailrocket_events",
        "missing_values",
        "warning",
        d["transactionid"].isna().sum(),
        "Nulls in transactionid",
        file_path
    )
    if item:
        issues.append(item)

    item = make_issue(
        "retailrocket_events",
        "duplicates",
        "warning",
        d.duplicated().sum(),
        "Duplicate rows",
        file_path
    )
    if item:
        issues.append(item)

    return issues


def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
```

```
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_
    if item:
        issues.append(item)
        return issues

    item = make_issue(
        "retailrocket_category_tree",
        "missing_values",
        "warning",
        d["parentid"].isna().sum(),
        "Nulls in parentid",
        file_path
    )
    if item:
        issues.append(item)

    return issues


def validate_raw_products(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["id", "title", "category", "price"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("dummyjson_products_raw", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_pat
    if item:
        issues.append(item)

    if "brand" in d.columns:
        item = make_issue(
            "dummyjson_products_raw",
            "missing_values",
            "warning",
            d["brand"].isna().sum(),
            "Nulls in brand",
            file_path
        )
        if item:
            issues.append(item)

    return issues


def validate_raw_categories(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    has_valid_category_field = any(col in d.columns for col in ["category", "slug", "name"])
    item = make_issue(
        "dummyjson_categories_raw",
        "schema",
        "error",
        0 if has_valid_category_field else 1,
        "Expected one of ['category', 'slug', 'name'] in categories payload",
        file_path
    )
    if item:
        issues.append(item)

    return issues


# ----------------------------
# Prepared validation functions
# ----------------------------
def validate_prepared_interactions(df, file_path):
    issues = []
    required = ["user_id", "item_id", "event", "event_weight", "event_ts"]
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("interactions_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path
    if item:
        issues.append(item)
        return issues

    item = make_issue("interactions_prepared", "duplicates", "warning", df.duplicated().sum(), "Duplicate rows", file_path)
    if item:
        issues.append(item)

    bad_txn = ((df["event"] == "transaction") & (df["transactionid"].isna())).sum()
    item = make_issue("interactions_prepared", "missing_values", "error", bad_txn, "Null transactionid for transaction events", fi
    if item:
        issues.append(item)

    return issues


def validate_prepared_products(df, file_path):
    issues = []
    required = ["product_id", "title", "category", "price"]
```

```
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("products_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    item = make_issue("products_prepared", "duplicates", "warning", df.duplicated().sum(), "Duplicate rows", file_path)
    if item:
        issues.append(item)

    return issues


def validate_prepared_categories(df, file_path):
    issues = []
    required = ["category"]
    missing_cols = [c for c in required if c not in df.columns]
    item = make_issue("categories_prepared", "schema", "error", len(missing_cols), f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    blanks = (df["category"].astype(str).str.strip() == "").sum()
    item = make_issue("categories_prepared", "missing_values", "error", blanks, "Blank category values", file_path)
    if item:
        issues.append(item)

    return issues


# ----------------------------
# Load the raw ingestion files
# ----------------------------
events_raw = pd.read_csv(files["events_csv"])
category_tree_raw = pd.read_csv(files["category_tree_csv"])
item_props_1 = pd.read_csv(files["item_properties_part1_csv"])
item_props_2 = pd.read_csv(files["item_properties_part2_csv"])
item_properties_raw = pd.concat([item_props_1, item_props_2], ignore_index=True)

with open(files["products_raw_json"], "r", encoding="utf-8") as f:
    products_payload = json.load(f)

with open(files["categories_raw_json"], "r", encoding="utf-8") as f:
    categories_payload = json.load(f)

products_raw = pd.json_normalize(products_payload.get("products", []), sep="_")

if isinstance(categories_payload, list):
    if len(categories_payload) > 0 and isinstance(categories_payload[0], dict):
        categories_raw = pd.json_normalize(categories_payload, sep="_")
    else:
        categories_raw = pd.DataFrame({"category": categories_payload})
elif isinstance(categories_payload, dict):
    categories_raw = pd.json_normalize(categories_payload, sep="_")
else:
    categories_raw = pd.DataFrame()

print("events_raw:", events_raw.shape)
print("category_tree_raw:", category_tree_raw.shape)
print("item_properties_raw:", item_properties_raw.shape)
print("products_raw:", products_raw.shape)
print("categories_raw:", categories_raw.shape)


# ----------------------------
# Quick profiling snapshot
# ----------------------------
profile_df(events_raw, "events_raw")
profile_df(category_tree_raw, "category_tree_raw")
profile_df(item_properties_raw, "item_properties_raw")
profile_df(products_raw, "products_raw")
profile_df(categories_raw, "categories_raw")


# ----------------------------
# Raw validation
# ----------------------------
raw_issues = []
raw_issues.extend(validate_raw_events(events_raw, files["events_csv"]))
raw_issues.extend(validate_raw_category_tree(category_tree_raw, files["category_tree_csv"]))
raw_issues.extend(validate_raw_products(products_raw, files["products_raw_json"]))
raw_issues.extend(validate_raw_categories(categories_raw, files["categories_raw_json"]))

raw_summaries = [
    summarize_validation("retailrocket_events", events_raw, [i for i in raw_issues if i["dataset"] == "retailrocket_events"]),
    summarize_validation("retailrocket_category_tree", category_tree_raw, [i for i in raw_issues if i["dataset"] == "retailrocket_
    summarize_validation("dummyjson_products_raw", products_raw, [i for i in raw_issues if i["dataset"] == "dummyjson_products_raw
    summarize_validation("dummyjson_categories_raw", categories_raw, [i for i in raw_issues if i["dataset"] == "dummyjson_categori
]

raw_issues_df, raw_summary_df, raw_issues_path, raw_summary_path = save_validation_reports(raw_issues, raw_summaries, "raw_validat

print("\nRAW VALIDATION ISSUES")
```

```
display(raw_issues_df)

print("\nRAW VALIDATION SUMMARY")
display(raw_summary_df)


# ----------------------------
# Clean and prepare Retailrocket interactions
# ----------------------------
events = events_raw.copy()

events.columns = [c.strip().lower() for c in events.columns]

required_event_cols = ["timestamp", "visitorid", "event", "itemid"]
missing_cols = [c for c in required_event_cols if c not in events.columns]
if missing_cols:
    raise ValueError(f"events.csv missing columns: {missing_cols}")

events = events.dropna(subset=["timestamp", "visitorid", "event", "itemid"]).copy()

events["timestamp"] = pd.to_numeric(events["timestamp"], errors="coerce")
events["visitorid"] = pd.to_numeric(events["visitorid"], errors="coerce")
events["itemid"] = pd.to_numeric(events["itemid"], errors="coerce")

events = events.dropna(subset=["timestamp", "visitorid", "itemid"]).copy()

events["event"] = events["event"].astype(str).str.strip().str.lower()
events = events[events["event"].isin(["view", "addtocart", "transaction"])].copy()

if "transactionid" not in events.columns:
    events["transactionid"] = np.nan

events["transactionid"] = pd.to_numeric(events["transactionid"], errors="coerce")
events["transactionid_missing"] = events["transactionid"].isna().astype(int)

bad_transactions = events[(events["event"] == "transaction") & (events["transactionid"].isna())].copy()
if not bad_transactions.empty:
    bad_transactions.to_csv(QUARANTINE_ROOT / f"invalid_transaction_rows_{RUN_TS}.csv", index=False)

events = events[~((events["event"] == "transaction") & (events["transactionid"].isna()))].copy()

events["event_ts"] = pd.to_datetime(events["timestamp"], unit="ms", errors="coerce")
events = events.dropna(subset=["event_ts"]).copy()

events["user_id"] = events["visitorid"].astype("int64")
events["item_id"] = events["itemid"].astype("int64")
events["event_date"] = events["event_ts"].dt.date
events["event_hour"] = events["event_ts"].dt.hour
events["event_dow"] = events["event_ts"].dt.day_name()

event_weight_map = {"view": 1.0, "addtocart": 3.0, "transaction": 5.0}
events["event_weight"] = events["event"].map(event_weight_map).astype(float)

events = events.drop_duplicates(subset=["timestamp", "user_id", "item_id", "event"]).copy()

interactions_prepared = events[
    [
        "user_id",
        "item_id",
        "event",
        "event_weight",
        "event_ts",
        "event_date",
        "event_hour",
        "event_dow",
        "transactionid",
        "transactionid_missing",
    ]
].reset_index(drop=True)

print(interactions_prepared.shape)
display(interactions_prepared.head())


# ----------------------------
# Prepare Retailrocket item attributes snapshot
# ----------------------------
category_tree = category_tree_raw.copy()
category_tree.columns = [c.strip().lower() for c in category_tree.columns]

if "parentid" in category_tree.columns:
    category_tree["is_root"] = category_tree["parentid"].isna().astype(int)
    category_tree["parentid"] = category_tree["parentid"].fillna(-1)

item_properties = item_properties_raw.copy()
item_properties.columns = [c.strip().lower() for c in item_properties.columns]

item_properties = item_properties.dropna(subset=["timestamp", "itemid", "property", "value"]).copy()
item_properties["timestamp"] = pd.to_numeric(item_properties["timestamp"], errors="coerce")
item_properties["itemid"] = pd.to_numeric(item_properties["itemid"], errors="coerce")
item_properties["prop_ts"] = pd.to_datetime(item_properties["timestamp"], unit="ms", errors="coerce")
item_properties = item_properties.dropna(subset=["timestamp", "itemid", "prop_ts"]).copy()
```

```
item_properties["item_id"] = item_properties["itemid"].astype("int64")
item_properties["property"] = item_properties["property"].astype(str).str.strip().str.lower()
item_properties["value"] = item_properties["value"].astype(str).str.strip()

latest_item_props = (
    item_properties
    .sort_values(["item_id", "property", "prop_ts"])
    .drop_duplicates(subset=["item_id", "property"], keep="last")
)

top_properties = (
    latest_item_props["property"]
    .value_counts()
    .head(25)
    .index
    .tolist()
)

item_snapshot = (
    latest_item_props[latest_item_props["property"].isin(top_properties)]
    .pivot(index="item_id", columns="property", values="value")
    .reset_index()
)

print("Top properties used:", len(top_properties))
print("item_snapshot:", item_snapshot.shape)
display(item_snapshot.head())


# ----------------------------
# Clean, encode, and normalize DummyJSON products
# ----------------------------
def minmax_scale(series):
    s = pd.to_numeric(series, errors="coerce")
    s_min = s.min()
    s_max = s.max()
    if pd.isna(s_min) or pd.isna(s_max) or s_min == s_max:
        return pd.Series(np.zeros(len(s)), index=s.index)
    return (s - s_min) / (s_max - s_min)


products = products_raw.copy()
products.columns = [c.strip().lower() for c in products.columns]

required_product_cols = ["id", "title", "category", "price"]
missing_product_cols = [c for c in required_product_cols if c not in products.columns]
if missing_product_cols:
    raise ValueError(f"products_raw.json missing fields: {missing_product_cols}")

products = products.drop_duplicates(subset=["id"]).copy()
products["id"] = pd.to_numeric(products["id"], errors="coerce")
products["price"] = pd.to_numeric(products.get("price"), errors="coerce")
products["rating"] = pd.to_numeric(products.get("rating"), errors="coerce")
products["stock"] = pd.to_numeric(products.get("stock"), errors="coerce")
products["discountpercentage"] = pd.to_numeric(products.get("discountpercentage"), errors="coerce")

products["title"] = products["title"].astype(str).str.strip()
products["category"] = products["category"].astype(str).str.strip().str.lower()

if "brand" not in products.columns:
    products["brand"] = "Unknown"
else:
    products["brand"] = products["brand"].fillna("Unknown").astype(str).str.strip()
    products.loc[products["brand"].eq(""), "brand"] = "Unknown"

products = products.dropna(subset=["id", "title", "category", "price"]).copy()
products = products[products["price"] >= 0].copy()

products["product_id"] = products["id"].astype("int64")
products["price_norm"] = minmax_scale(products["price"])
products["rating_norm"] = minmax_scale(products["rating"])
products["stock_norm"] = minmax_scale(products["stock"])
products["discount_norm"] = minmax_scale(products["discountpercentage"])

category_dummies = pd.get_dummies(products["category"], prefix="cat", dtype=int)
brand_dummies = pd.get_dummies(products["brand"], prefix="brand", dtype=int)

products_prepared = pd.concat(
    [
        products[
            [
                "product_id",
                "title",
                "category",
                "brand",
                "price",
                "rating",
                "stock",
                "discountpercentage",
                "price
```

[TRUNCATED FOR PDF SIZE]

Code: src\04-features\materialize_features.ipynb

```
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd

# ----------------------------
# Project paths and setup
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
#PROJECT_ROOT = Path(__file__).resolve().parents[1]
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
FEATURE_ROOT = PROJECT_ROOT / "data" / "featured" / "v1"

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

USER_FEATURES_PATH = FEATURE_ROOT / "user_features.parquet"
ITEM_FEATURES_PATH = FEATURE_ROOT / "item_features.parquet"
INTERACTION_FEATURES_PATH = FEATURE_ROOT / "interaction_features.parquet"


def min_max_scale(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=series.index)

    min_val = s.min()
    max_val = s.max()

    if pd.isna(min_val) or pd.isna(max_val):
        return pd.Series(np.nan, index=series.index)

    if min_val == max_val:
        return pd.Series(0.0, index=series.index)

    return (s - min_val) / (max_val - min_val)


def ensure_required_columns(df: pd.DataFrame, required: list[str], df_name: str) -> None:
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{df_name} is missing required columns: {missing}")


def load_prepared_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    if not INTERACTIONS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {INTERACTIONS_PATH}")

    if not PRODUCTS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {PRODUCTS_PATH}")

    interactions = pd.read_csv(INTERACTIONS_PATH)
    products = pd.read_csv(PRODUCTS_PATH)

    ensure_required_columns(
        interactions,
        ["user_id", "item_id", "event", "event_weight", "event_ts", "event_date"],
        "interactions_prepared.csv",
    )
    ensure_required_columns(
        products,
        ["category"],
        "products_prepared.csv",
    )

    interactions["user_id"] = pd.to_numeric(interactions["user_id"], errors="coerce").astype("Int64")
    interactions["item_id"] = pd.to_numeric(interactions["item_id"], errors="coerce").astype("Int64")
    interactions["event"] = interactions["event"].astype(str).str.strip().str.lower()
    interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce")
    interactions["event_ts"] = pd.to_datetime(interactions["event_ts"], errors="coerce")
    interactions["event_date"] = pd.to_datetime(interactions["event_date"], errors="coerce")

    interactions = interactions.dropna(subset=["user_id", "item_id", "event", "event_weight", "event_ts"])
    interactions["user_id"] = interactions["user_id"].astype("int64")
    interactions["item_id"] = interactions["item_id"].astype("int64")

    products.columns = [c.strip() for c in products.columns]

    return interactions, products


def normalize_products(products: pd.DataFrame) -> pd.DataFrame:
    products = products.copy()

    if "item_id" not in products.columns:
```

```
        if "product_id" in products.columns:
            products["item_id"] = pd.to_numeric(products["product_id"], errors="coerce")
        elif "id" in products.columns:
            products["item_id"] = pd.to_numeric(products["id"], errors="coerce")

    if "item_id" not in products.columns:
        raise ValueError("products_prepared.csv must contain one of: item_id, product_id, or id")

    products["item_id"] = pd.to_numeric(products["item_id"], errors="coerce")
    products = products.dropna(subset=["item_id"]).copy()
    products["item_id"] = products["item_id"].astype("int64")

    if "category" in products.columns:
        products["category"] = (
            products["category"]
            .astype(str)
            .str.strip()
            .str.lower()
        )
    else:
        products["category"] = "unknown"

    if "price_norm" not in products.columns:
        products["price_norm"] = min_max_scale(products["price"]) if "price" in products.columns else 0.0

    if "rating_norm" not in products.columns:
        if "rating" in products.columns:
            products["rating_norm"] = min_max_scale(products["rating"])
        else:
            products["rating_norm"] = 0.0

    if "stock_norm" not in products.columns:
        if "stock" in products.columns:
            products["stock_norm"] = min_max_scale(products["stock"])
        else:
            products["stock_norm"] = 0.0

    keep_cols = ["item_id", "price_norm", "rating_norm", "stock_norm", "category"]
    return products[keep_cols].drop_duplicates(subset=["item_id"]).reset_index(drop=True)


def build_user_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    user_features = (
        interactions.groupby("user_id")
        .agg(
            user_total_interactions=("user_id", "size"),
            user_view_count=("event", lambda s: (s == "view").sum()),
            user_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            user_transaction_count=("event", lambda s: (s == "transaction").sum()),
            user_avg_event_weight=("event_weight", "mean"),
            user_active_days=("event_date", lambda s: s.dt.date.nunique()),
            last_event_ts=("event_ts", "max"),
        )
        .reset_index()
    )

    user_features["user_last_seen_recency_days"] = (
        (max_ts - user_features["last_event_ts"]).dt.total_seconds() / 86400.0
    )

    user_features = user_features.drop(columns=["last_event_ts"])

    return user_features


def build_item_features(interactions: pd.DataFrame, products: pd.DataFrame) -> pd.DataFrame:
    item_features = (
        interactions.groupby("item_id")
        .agg(
            item_total_interactions=("item_id", "size"),
            item_view_count=("event", lambda s: (s == "view").sum()),
            item_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            item_transaction_count=("event", lambda s: (s == "transaction").sum()),
            item_avg_event_weight=("event_weight", "mean"),
        )
        .reset_index()
    )

    item_features["item_popularity_rank"] = (
        item_features["item_total_interactions"]
        .rank(method="dense", ascending=False)
        .astype("int64")
    )

    products_norm = normalize_products(products)

    item_features = item_features.merge(products_norm, on="item_id", how="left")

    item_features["category"] = item_features["category"].fillna("unknown")
    for col in ["price_norm", "rating_norm", "stock_norm"]:
        item_features[col] = item_features[col].fillna(0.0)
```

```
    return item_features


def build_interaction_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    interaction_features = (
        interactions.groupby(["user_id", "item_id"])
        .agg(
            user_item_event_weight_sum=("event_weight", "sum"),
            user_item_interaction_count=("item_id", "size"),
            last_event_ts=("event_ts", "max"),
        )
        .reset_index()
    )

    interaction_features["user_item_last_event_recency_days"] = (
        (max_ts - interaction_features["last_event_ts"]).dt.total_seconds() / 86400.0
    )

    interaction_features = interaction_features.drop(columns=["last_event_ts"])

    return interaction_features


def save_feature_sets(
    user_features: pd.DataFrame,
    item_features: pd.DataFrame,
    interaction_features: pd.DataFrame,
) -> None:
    FEATURE_ROOT.mkdir(parents=True, exist_ok=True)

    user_features.to_parquet(USER_FEATURES_PATH, index=False)
    item_features.to_parquet(ITEM_FEATURES_PATH, index=False)
    interaction_features.to_parquet(INTERACTION_FEATURES_PATH, index=False)

    print(f"Saved: {USER_FEATURES_PATH}")
    print(f"Saved: {ITEM_FEATURES_PATH}")
    print(f"Saved: {INTERACTION_FEATURES_PATH}")


def main() -> None:
    print(f"Project root: {PROJECT_ROOT}")
    print(f"Loading: {INTERACTIONS_PATH}")
    print(f"Loading: {PRODUCTS_PATH}")

    interactions, products = load_prepared_data()

    user_features = build_user_features(interactions)
    item_features = build_item_features(interactions, products)
    interaction_features = build_interaction_features(interactions)

    save_feature_sets(user_features, item_features, interaction_features)

    print("\nShapes:")
    print(f"user_features: {user_features.shape}")
    print(f"item_features: {item_features.shape}")
    print(f"interaction_features: {interaction_features.shape}")

    print("\nSample user_features:")
    print(user_features.head().to_string(index=False))

    print("\nSample item_features:")
    print(item_features.head().to_string(index=False))

    print("\nSample interaction_features:")
    print(interaction_features.head().to_string(index=False))


if __name__ == "__main__":
    main()
```

Code: src\04-features\registry_loader copy.ipynb

```
from pathlib import Path
import yaml
from pathlib import Path
print(Path.cwd())

def load_feature_registry(registry_path="../../feature_store/registry/feature_registry.yaml"):
    registry_path = Path(registry_path)
    with open(registry_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)

def get_feature_sets(registry, mode="training", version="v1"):
    allowed_sets = set(registry["governance"]["retrieval_rules"][mode]["include_feature_sets"])
    selected = []

    for fs in registry["feature_sets"]:
        if fs["name"] in allowed_sets and fs["version"] == version:
```

```
            selected.append(fs)

    return selected

def list_feature_names(registry, mode="training", version="v1"):
    feature_sets = get_feature_sets(registry, mode=mode, version=version)
    output = {}

    for fs in feature_sets:
        output[fs["name"]] = [feature["name"] for feature in fs["features"]]

    return output

if __name__ == "__main__":
    registry = load_feature_registry()
    print("Training features:")
    print(list_feature_names(registry, mode="training", version="v1"))
    print("\nInference features:")
    print(list_feature_names(registry, mode="inference", version="v1"))
```

Code: src\04-features\registry_loader.ipynb

```
from __future__ import annotations

from pathlib import Path
from typing import Any, Dict, List, Optional, Union

import pandas as pd
import yaml


def find_project_root(start: Optional[Path] = None) -> Path:
    current = (start or Path.cwd()).resolve()
    while not (current / "src").exists() and current != current.parent:
        current = current.parent
    return current


PROJECT_ROOT = find_project_root()
DEFAULT_REGISTRY_PATH = PROJECT_ROOT / "feature_store" / "registry" / "feature_registry.yaml"


class FeatureRegistryError(Exception):
    pass


def normalize_registry_path(registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH) -> Path:
    path = Path(registry_path)

    if path.is_absolute():
        return path.resolve()

    # If caller passes something like "../../feature_store/..."
    # resolve it from the current working directory first.
    cwd_resolved = (Path.cwd() / path).resolve()
    if cwd_resolved.exists():
        return cwd_resolved

    # Otherwise resolve relative to repo root.
    project_resolved = (PROJECT_ROOT / path).resolve()
    return project_resolved


def load_feature_registry(
    registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH
) -> Dict[str, Any]:
    registry_path = normalize_registry_path(registry_path)

    if not registry_path.exists():
        raise FileNotFoundError(
            f"Registry file not found: {registry_path}\n"
            f"PROJECT_ROOT resolved to: {PROJECT_ROOT}"
        )

    with open(registry_path, "r", encoding="utf-8") as f:
        registry = yaml.safe_load(f)

    if not isinstance(registry, dict):
        raise FeatureRegistryError("Registry YAML did not load as a dictionary.")

    return registry


def _entity_map(registry: Dict[str, Any]) -> Dict[str, Dict[str, Any]]:
    return {entity["name"]: entity for entity in registry.get("entities", [])}


def validate_registry(registry: Dict[str, Any]) -> List[str]:
    errors: List[str] = []
```

```
    required_top_level = [
        "registry_name",
        "registry_version",
        "entities",
        "feature_sets",
        "governance",
    ]
    for key in required_top_level:
        if key not in registry:
            errors.append(f"Missing top-level key: {key}")

    entity_lookup = _entity_map(registry)

    for entity_name, entity_def in entity_lookup.items():
        has_join_key = "join_key" in entity_def
        has_join_keys = "join_keys" in entity_def

        if not (has_join_key or has_join_keys):
            errors.append(f"Entity '{entity_name}' must define 'join_key' or 'join_keys'.")

        if has_join_key and has_join_keys:
            errors.append(f"Entity '{entity_name}' must define only one of 'join_key' or 'join_keys'.")

    feature_sets = registry.get("feature_sets", [])
    for fs in feature_sets:
        fs_name = fs.get("name", "<unknown>")
        entity_name = fs.get("entity")
        if entity_name not in entity_lookup:
            errors.append(f"Feature set '{fs_name}' references unknown entity '{entity_name}'.")

        for key in ["version", "storage_path", "features"]:
            if key not in fs:
                errors.append(f"Feature set '{fs_name}' missing required key '{key}'.")

        if "features" in fs and not fs["features"]:
            errors.append(f"Feature set '{fs_name}' must include at least one feature.")

        for feature in fs.get("features", []):
            for feature_key in ["name", "dtype", "source_column", "transformation"]:
                if feature_key not in feature:
                    errors.append(
                        f"Feature set '{fs_name}' has feature missing '{feature_key}'."
                    )

    retrieval_rules = registry.get("governance", {}).get("retrieval_rules", {})
    for mode in ["training", "inference"]:
        if mode not in retrieval_rules:
            errors.append(f"Missing retrieval rules for mode '{mode}'.")
            continue

        include_sets = retrieval_rules[mode].get("include_feature_sets", [])
        known_sets = {fs.get("name") for fs in feature_sets}
        for fs_name in include_sets:
            if fs_name not in known_sets:
                errors.append(
                    f"Retrieval mode '{mode}' includes unknown feature set '{fs_name}'."
                )

    return errors


def get_feature_sets(
    registry: Dict[str, Any],
    mode: str = "training",
    version: Optional[str] = None
) -> List[Dict[str, Any]]:
    retrieval_rules = registry["governance"]["retrieval_rules"]
    allowed_sets = set(retrieval_rules[mode]["include_feature_sets"])

    if version is None:
        version = registry.get("registry_version")

    selected = []
    for fs in registry["feature_sets"]:
        if fs["name"] in allowed_sets and fs["version"] == version:
            selected.append(fs)

    return selected


def list_feature_names(
    registry: Dict[str, Any],
    mode: str = "training",
    version: Optional[str] = None
) -> Dict[str, List[str]]:
    feature_sets = get_feature_sets(registry, mode=mode, version=version)
    return {
        fs["name"]: [feature["name"] for feature in fs["features"]]
        for fs in feature_sets
    }


def get_join_keys_for_entity(
```

```
    registry: Dict[str, Any],
    entity_name: str
) -> List[str]:
    entity_lookup = _entity_map(registry)

    if entity_name not in entity_lookup:
        raise FeatureRegistryError(f"Unknown entity '{entity_name}'.")

    entity_def = entity_lookup[entity_name]

    if "join_key" in entity_def:
        return [entity_def["join_key"]]

    return entity_def["join_keys"]


def resolve_storage_path(
    feature_set: Dict[str, Any],
    registry_path: Union[str, Path]
) -> Path:
    registry_path = normalize_registry_path(registry_path)
    storage_path = Path(feature_set["storage_path"])

    if storage_path.is_absolute():
        return storage_path.resolve()

    return (PROJECT_ROOT / storage_path).resolve()


def load_feature_data(
    feature_set: Dict[str, Any],
    registry_path: Union[str, Path]
) -> pd.DataFrame:
    storage_path = resolve_storage_path(feature_set, registry_path)

    if not storage_path.exists():
        raise FileNotFoundError(
            f"Feature data file not found for feature set '{feature_set['name']}': {storage_path}"
        )

    suffix = storage_path.suffix.lower()
    if suffix == ".parquet":
        return pd.read_parquet(storage_path)
    if suffix == ".csv":
        return pd.read_csv(storage_path)

    raise ValueError(
        f"Unsupported storage format '{suffix}' for feature set '{feature_set['name']}'. "
        "Use .parquet or .csv."
    )


def get_required_columns(
    registry: Dict[str, Any],
    feature_set: Dict[str, Any]
) -> List[str]:
    join_keys = get_join_keys_for_entity(registry, feature_set["entity"])
    feature_names = [f["name"] for f in feature_set["features"]]
    return join_keys + feature_names


def validate_feature_data_columns(
    registry: Dict[str, Any],
    feature_set: Dict[str, Any],
    df: pd.DataFrame
) -> None:
    required_cols = get_required_columns(registry, feature_set)
    missing_cols = [col for col in required_cols if col not in df.columns]

    if missing_cols:
        raise FeatureRegistryError(
            f"Feature data for '{feature_set['name']}' is missing required columns: {missing_cols}"
        )


def retrieve_feature_set(
    registry: Dict[str, Any],
    feature_set: Dict[str, Any],
    entity_df: pd.DataFrame,
    registry_path: Union[str, Path]
) -> pd.DataFrame:
    join_keys = get_join_keys_for_entity(registry, feature_set["entity"])
    feature_df = load_feature_data(feature_set, registry_path)
    validate_feature_data_columns(registry, feature_set, feature_df)

    for key in join_keys:
        if key not in entity_df.columns:
            raise FeatureRegistryError(
                f"Entity input is missing join key '{key}' required for feature set '{feature_set['name']}'."
            )

    selected_cols = get_required_columns(registry, feature_set)
    feature_df = feature_df[selected_cols].drop_duplicates(subset=join_keys)
```

```
    merged = entity_df.merge(feature_df, on=join_keys, how="left")
    return merged


def retrieve_features(
    registry: Dict[str, Any],
    entity_df: pd.DataFrame,
    mode: str = "training",
    version: Optional[str] = None,
    registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH
) -> pd.DataFrame:
    feature_sets = get_feature_sets(registry, mode=mode, version=version)
    result = entity_df.copy()

    for fs in feature_sets:
        result = retrieve_feature_set(registry, fs, result, registry_path)

    return result


def get_training_dataset(
    registry: Dict[str, Any],
    entity_df: pd.DataFrame,
    label_column: Optional[str] = None,
    version: Optional[str] = None,
    registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH
) -> pd.DataFrame:
    training_df = retrieve_features(
        registry=registry,
        entity_df=entity_df,
        mode="training",
        version=version,
        registry_path=registry_path,
    )

    if label_column and label_column not in training_df.columns:
        raise FeatureRegistryError(
            f"Label column '{label_column}' not found after training retrieval."
        )

    return training_df


def get_inference_dataset(
    registry: Dict[str, Any],
    entity_df: pd.DataFrame,
    version: Optional[str] = None,
    registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH
) -> pd.DataFrame:
    return retrieve_features(
        registry=registry,
        entity_df=entity_df,
        mode="inference",
        version=version,
        registry_path=registry_path,
    )


def describe_registry(
    registry: Dict[str, Any],
    mode: str = "training",
    version: Optional[str] = None
) -> pd.DataFrame:
    rows = []
    for fs in get_feature_sets(registry, mode=mode, version=version):
        join_keys = get_join_keys_for_entity(registry, fs["entity"])
        for feature in fs["features"]:
            rows.append({
                "feature_set": fs["name"],
                "entity": fs["entity"],
                "join_keys": ",".join(join_keys),
                "feature_name": feature["name"],
                "dtype": feature["dtype"],
                "source_column": feature["source_column"],
                "transformation": feature["transformation"],
                "storage_path": fs["storage_path"],
                "version": fs["version"],
            })
    return pd.DataFrame(rows)


def demo(
    registry_path: Union[str, Path] = DEFAULT_REGISTRY_PATH,
    version: Optional[str] = None
) -> None:
    resolved_registry_path = normalize_registry_path(registry_path)

    print("PROJECT_ROOT:", PROJECT_ROOT)
    print("CWD:", Path.cwd().resolve())
    print("REGISTRY_PATH:", resolved_registry_path)

    registry = load_feature_registry(resolved_registry_path)
```

```
    errors = validate_registry(registry)
    if errors:
        raise FeatureRegistryError("Registry validation failed:\n- " + "\n- ".join(errors))

    if version is None:
        version = registry.get("registry_version", "v1")

    print(f"Loaded registry: {registry['registry_name']} ({version})")
    print("\nTraining features:")
    print(list_feature_names(registry, mode="training", version=version))

    print("\nInference features:")
    print(list_feature_names(registry, mode="inference", version=version))

    print("\nFeature catalog:")
    print(describe_registry(registry, mode="training", version=version).head(20).to_string(index=False))

    training_entities = pd.DataFrame({
        "user_id": [101, 102, 103],
        "item_id": [5001, 5002, 5003],
    })

    inference_entities = pd.DataFrame({
        "user_id": [101, 102],
        "item_id": [5001, 5002],
    })

    print("\nTraining entity input:")
    print(training_entities.to_string(index=False))

    try:
        training_dataset = get_training_dataset(
            registry=registry,
            entity_df=training_entities,
            version=version,
            registry_path=resolved_registry_path,
        )
        print("\nRetrieved training dataset:")
        print(training_dataset.head().to_string(index=False))
    except Exception as e:
        print(f"\nTraining retrieval demo could not complete: {e}")

    print("\nInference entity input:")
    print(inference_entities.to_string(index=False))

    try:
        inference_dataset = get_inference_dataset(
            registry=registry,
            entity_df=inference_entities,
            version=version,
            registry_path=resolved_registry_path,
        )
        print("\nRetrieved inference dataset:")
        print(inference_dataset.head().to_string(index=False))
    except Exception as e:
        print(f"\nInference retrieval demo could not complete: {e}")


if __name__ == "__main__":
    demo()
```

Code: src\05-training\comparision.ipynb

```
# Code Generated by Sidekick is for learning and experimentation purposes only.
import json
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
from pathlib import Path

# ----------------------------
# 1) Locate project folders
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

REPORT_ROOT = PROJECT_ROOT / "reports"
REPORT_ROOT.mkdir(parents=True, exist_ok=True)

SVD_METRICS_PATH = REPORT_ROOT / "svd_metrics.json"
ITEM_CF_METRICS_PATH = REPORT_ROOT / "item_cf_metrics.json"
CONTENT_METRICS_PATH = REPORT_ROOT / "content_based_metrics.json"

OUTPUT_CSV_PATH = REPORT_ROOT / "model_comparison_table_3_models.csv"
OUTPUT_PDF_PATH = REPORT_ROOT / "model_comparison_table_3_models.pdf"

# ----------------------------
# 2) Load metrics
# ----------------------------
```

```
required_files = [
    SVD_METRICS_PATH,
    ITEM_CF_METRICS_PATH,
    CONTENT_METRICS_PATH,
]

missing_files = [str(p) for p in required_files if not p.exists()]
if missing_files:
    raise FileNotFoundError(
        "Missing metrics file(s):\n- " + "\n- ".join(missing_files)
    )

with open(SVD_METRICS_PATH, "r", encoding="utf-8") as f:
    svd_metrics = json.load(f)

with open(ITEM_CF_METRICS_PATH, "r", encoding="utf-8") as f:
    item_cf_metrics = json.load(f)

with open(CONTENT_METRICS_PATH, "r", encoding="utf-8") as f:
    content_metrics = json.load(f)

# ----------------------------
# 3) Build comparison table
# ----------------------------
metric_order = [
    "precision_at_5",
    "recall_at_5",
    "ndcg_at_5",
    "evaluated_users_at_5",
    "precision_at_10",
    "recall_at_10",
    "ndcg_at_10",
    "evaluated_users_at_10",
    "runtime_minutes",
    "catalog_size",
    "catalog_size_used",
    "feature_vocab_size",
    "explained_variance_ratio_sum",
    "overlap_items",
    "eligible_users",
]

comparison_rows = []
for metric in metric_order:
    comparison_rows.append({
        "Metric": metric,
        "SVD": svd_metrics.get(metric, "NA"),
        "Item-Based CF": item_cf_metrics.get(metric, "NA"),
        "Content-Based": content_metrics.get(metric, "NA"),
    })

comparison_df = pd.DataFrame(comparison_rows)

# Keep only rows where at least one model has a real value
comparison_df = comparison_df[
    ~(
        (comparison_df["SVD"] == "NA") &
        (comparison_df["Item-Based CF"] == "NA") &
        (comparison_df["Content-Based"] == "NA")
    )
].copy()

# ----------------------------
# 4) Pretty formatting
# ----------------------------
display_df = comparison_df.copy()

def fmt_value(x):
    if isinstance(x, (int, float)) and not isinstance(x, bool):
        if abs(float(x)) >= 1000:
            return f"{x:,.0f}"
        return f"{x:.4f}"
    return str(x)

for col in ["SVD", "Item-Based CF", "Content-Based"]:
    display_df[col] = display_df[col].apply(fmt_value)

display_df["Metric"] = display_df["Metric"].replace({
    "precision_at_5": "Precision@5",
    "recall_at_5": "Recall@5",
    "ndcg_at_5": "NDCG@5",
    "evaluated_users_at_5": "Evaluated Users@5",
    "precision_at_10": "Precision@10",
    "recall_at_10": "Recall@10",
    "ndcg_at_10": "NDCG@10",
    "evaluated_users_at_10": "Evaluated Users@10",
    "runtime_minutes": "Runtime (Minutes)",
    "catalog_size": "Catalog Size",
    "catalog_size_used": "Catalog Size Used",
    "feature_vocab_size": "Feature Vocab Size",
    "explained_variance_ratio_sum": "Explained Variance Ratio Sum",
    "overlap_items": "Overlap Items",
    "eligible_users": "Eligible Users",
```

```
})

# ----------------------------
# 5) Save CSV
# ----------------------------
comparison_df.to_csv(OUTPUT_CSV_PATH, index=False)

# ----------------------------
# 6) Export table to PDF
# ----------------------------
fig_height = max(5, 0.55 * len(display_df) + 2.0)
fig, ax = plt.subplots(figsize=(13, fig_height))
ax.axis("off")

table = ax.table(
    cellText=display_df.values,
    colLabels=display_df.columns,
    cellLoc="center",
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(9.5)
table.scale(1, 1.45)

# Header styling
for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(weight="bold", color="black")
        cell.set_facecolor("#D9EAD3")
    else:
        cell.set_facecolor("white")

plt.title("Model Comparison: SVD vs Item-CF vs Content-Based", fontsize=14, weight="bold", pad=18)
plt.tight_layout()

with PdfPages(OUTPUT_PDF_PATH) as pdf:
    pdf.savefig(fig, bbox_inches="tight")

plt.show()
plt.close(fig)

print("Updated comparison outputs saved:")
print("-", OUTPUT_CSV_PATH)
print("-", OUTPUT_PDF_PATH)

print("\nPreview:")
print(display_df.to_string(index=False))
```

Code: src\05-training\content-based-training.ipynb

```
import json
import pickle
import time
from pathlib import Path

import mlflow
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel

start = time.time()

# ----------------------------
# 0) Project / MLflow setup
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_ROOT = PROJECT_ROOT / "data"
PREP_ROOT = DATA_ROOT / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports"
ARTIFACT_ROOT = PROJECT_ROOT / "models" / "artifacts"
MLRUNS_ROOT = PROJECT_ROOT / "mlruns"

REPORT_ROOT.mkdir(parents=True, exist_ok=True)
ARTIFACT_ROOT.mkdir(parents=True, exist_ok=True)
MLRUNS_ROOT.mkdir(parents=True, exist_ok=True)

tracking_uri = f"file:///{MLRUNS_ROOT.resolve().as_posix()}"
mlflow.set_tracking_uri(tracking_uri)
mlflow.set_experiment("recomart_content_based")

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

# ----------------------------
# 1) Load data
```

```
# ----------------------------
interactions = pd.read_csv(INTERACTIONS_PATH)
products = pd.read_csv(PRODUCTS_PATH)

interactions.columns = [c.strip().lower() for c in interactions.columns]
products.columns = [c.strip().lower() for c in products.columns]

required_interaction_cols = {"user_id", "item_id", "event_weight"}
missing_i = required_interaction_cols - set(interactions.columns)
if missing_i:
    raise ValueError(f"Missing required interaction columns: {missing_i}")

# flexible product id detection
product_id_col = None
for c in ["product_id", "item_id", "id"]:
    if c in products.columns:
        product_id_col = c
        break

if product_id_col is None:
    raise ValueError("Could not find a product ID column in products_prepared.csv")

# normalize IDs to avoid int/str mismatches
interactions["user_id"] = interactions["user_id"].astype(str).str.strip()
interactions["item_id"] = interactions["item_id"].astype(str).str.strip()
products[product_id_col] = products[product_id_col].astype(str).str.strip()

# numeric weight cleanup
interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce").fillna(0.0)

print("Loaded interactions:", interactions.shape)
print("Loaded products:", products.shape)
print("Using product ID column:", product_id_col)

# ----------------------------
# 2) Aggregate interactions
# ----------------------------
interactions = (
    interactions.groupby(["user_id", "item_id"], as_index=False)["event_weight"]
    .sum()
)

# ----------------------------
# 3) Prepare product content features
# ----------------------------
products = products.drop_duplicates(subset=[product_id_col]).copy()

candidate_text_cols = [
    "title", "name", "product_name", "category", "brand",
    "description", "desc", "subcategory", "color"
]
available_text_cols = [c for c in candidate_text_cols if c in products.columns]

if not available_text_cols:
    raise ValueError("No usable text columns found in products_prepared.csv for content-based features.")

for c in available_text_cols:
    products[c] = products[c].fillna("").astype(str).str.lower().str.strip()

# optional numeric feature: price bucket
if "price" in products.columns:
    products["price"] = pd.to_numeric(products["price"], errors="coerce")
    non_null_price = products["price"].notna().sum()
    if non_null_price >= 5:
        products["price_bucket"] = pd.qcut(
            products["price"].rank(method="first"),
            q=min(5, products["price"].nunique()),
            labels=False,
            duplicates="drop"
        ).astype("Int64").astype(str)
    else:
        products["price_bucket"] = ""
else:
    products["price_bucket"] = ""

products["content_text"] = products[available_text_cols].agg(" ".join, axis=1).str.strip()
products["content_text"] = (
    products["content_text"] + " pricebucket_" + products["price_bucket"].fillna("")
).str.strip()

products = products[[product_id_col, "content_text"] + [c for c in available_text_cols if c not in [product_id_col]]].copy()
products = products[products["content_text"].str.len() > 0].copy()

# ----------------------------
# 4) OVERLAP FILTER FIRST (critical fix)
# ----------------------------
catalog_item_ids = set(products[product_id_col])
overlap_items = set(interactions["item_id"]) & catalog_item_ids

print("\nOverlap diagnostics")
print("Total unique interaction items:", interactions["item_id"].nunique())
print("Total catalog items:", len(catalog_item_ids))
print("Overlapping items:", len(overlap_items))
```

```
interactions_cb = interactions[interactions["item_id"].isin(overlap_items)].copy()
products_cb = products[products[product_id_col].isin(overlap_items)].copy()

print("Rows after item-overlap filter:", len(interactions_cb))
print("Users after item-overlap filter:", interactions_cb["user_id"].nunique())
print("Items after item-overlap filter:", interactions_cb["item_id"].nunique())

# Keep only users with enough overlap for leave-one-out
user_item_counts = interactions_cb.groupby("user_id")["item_id"].nunique()
eligible_users = user_item_counts[user_item_counts >= 2].index
interactions_cb = interactions_cb[interactions_cb["user_id"].isin(eligible_users)].copy()

print("Eligible users with >=2 overlapped items:", interactions_cb["user_id"].nunique())

if interactions_cb.empty:
    raise ValueError("No usable overlap remains after filtering. Expand product coverage or inspect item ID mapping.")

# ----------------------------
# 5) Leave-one-out split AFTER overlap filtering
# ----------------------------
rng = np.random.RandomState(42)
interactions_cb["_rand"] = rng.rand(len(interactions_cb))
test_idx = interactions_cb.groupby("user_id")["_rand"].idxmin()

test_df = interactions_cb.loc[test_idx, ["user_id", "item_id", "event_weight"]].copy()
train_df = interactions_cb.drop(index=test_idx).copy()

# safety check: keep only users with at least 1 training item
train_counts = train_df.groupby("user_id")["item_id"].nunique()
valid_users = train_counts[train_counts >= 1].index
train_df = train_df[train_df["user_id"].isin(valid_users)].copy()
test_df = test_df[test_df["user_id"].isin(valid_users)].copy()

print("\nSplit diagnostics")
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("Train users:", train_df["user_id"].nunique())
print("Test users:", test_df["user_id"].nunique())

if test_df.empty:
    raise ValueError("Test set is empty after filtering. Cannot evaluate metrics.")

# ----------------------------
# 6) Build TF-IDF matrix
# ----------------------------
products_cb = products_cb.drop_duplicates(subset=[product_id_col]).copy()
products_cb = products_cb.sort_values(product_id_col).reset_index(drop=True)

vectorizer = TfidfVectorizer(
    stop_words="english",
    min_df=1,
    ngram_range=(1, 2)
)
tfidf_matrix = vectorizer.fit_transform(products_cb["content_text"])

item_ids = products_cb[product_id_col].astype(str).tolist()
item_to_idx = {item_id: idx for idx, item_id in enumerate(item_ids)}
idx_to_item = np.array(item_ids)

print("TF-IDF matrix:", tfidf_matrix.shape)
print("Feature vocab size:", len(vectorizer.vocabulary_))

# ----------------------------
# 7) Build user history
# ----------------------------
train_df = train_df[train_df["item_id"].isin(item_to_idx)].copy()
test_df = test_df[test_df["item_id"].isin(item_to_idx)].copy()

if test_df.empty:
    raise ValueError("All test items were removed after TF-IDF item alignment.")

user_history = (
    train_df.groupby("user_id")[["item_id", "event_weight"]]
    .apply(lambda g: list(zip(g["item_id"].tolist(), g["event_weight"].tolist())))
    .to_dict()
)

# ----------------------------
# 8) Recommendation function
# ----------------------------
def recommend_content(user_id, k=10):
    if user_id not in user_history:
        return []

    history = user_history[user_id]
    valid_history = [(item, wt) for item, wt in history if item in item_to_idx]

    if not valid_history:
        return []

    seen_item_ids = [item for item, _ in valid_history]
    seen_indices = np.array([item_to_idx[item] for item in seen_item_ids], dtype=np.int32)
    seen_weights = np.array([float(wt) for _, wt in valid_history], dtype=np.float32)
```

```
    # weighted user profile
    user_profile = np.average(tfidf_matrix[seen_indices].toarray(), axis=0, weights=seen_weights)
    scores = linear_kernel([user_profile], tfidf_matrix).ravel()

    # remove seen items
    scores[seen_indices] = -np.inf

    finite_mask = np.isfinite(scores)
    candidate_count = int(finite_mask.sum())
    if candidate_count == 0:
        return []

    top_n = min(k, candidate_count)
    top_idx = np.argpartition(scores, -top_n)[-top_n:]
    top_idx = top_idx[np.argsort(scores[top_idx])[::-1]]
    top_idx = top_idx[np.isfinite(scores[top_idx])]

    return idx_to_item[top_idx].tolist()

sample_user = train_df["user_id"].iloc[0]
sample_recs = recommend_content(sample_user, k=5)
print("\nSample user:", sample_user)
print("Top-5 recommendations:", sample_recs)

# ----------------------------
# 9) Evaluation
# ----------------------------
def precision_recall_ndcg_at_k(eval_test_df, k=10, max_eval_users=10000):
    user_truth = eval_test_df.groupby("user_id")["item_id"].apply(set).to_dict()
    eval_users = list(user_truth.keys())[:max_eval_users]

    precisions, recalls, ndcgs = [], [], []

    for user_id in eval_users:
        recs = recommend_content(user_id, k=k)
        if not recs:
            continue

        true_items = user_truth[user_id]
        hits = np.array([1.0 if item in true_items else 0.0 for item in recs], dtype=np.float32)

        precision = float(hits.sum() / k)
        recall = float(hits.sum() / len(true_items))

        discounts = 1.0 / np.log2(np.arange(2, len(hits) + 2))
        dcg = float((hits * discounts).sum())

        ideal_len = min(len(true_items), k)
        idcg = float((np.ones(ideal_len) / np.log2(np.arange(2, ideal_len + 2))).sum())
        ndcg = dcg / idcg if idcg > 0 else 0.0

        precisions.append(precision)
        recalls.append(recall)
        ndcgs.append(ndcg)

    return {
        f"precision_at_{k}": float(np.mean(precisions)) if precisions else 0.0,
        f"recall_at_{k}": float(np.mean(recalls)) if recalls else 0.0,
        f"ndcg_at_{k}": float(np.mean(ndcgs)) if ndcgs else 0.0,
        f"evaluated_users_at_{k}": int(len(precisions)),
    }

metrics_5 = precision_recall_ndcg_at_k(test_df, k=5, max_eval_users=10000)
metrics_10 = precision_recall_ndcg_at_k(test_df, k=10, max_eval_users=10000)

runtime_minutes = (time.time() - start) / 60.0

all_metrics = {
    **metrics_5,
    **metrics_10,
    "runtime_minutes": float(runtime_minutes),
    "catalog_size_used": int(len(products_cb)),
    "feature_vocab_size": int(len(vectorizer.vocabulary_)),
    "overlap_items": int(len(overlap_items)),
    "eligible_users": int(train_df["user_id"].nunique()),
}

print("\nContent-Based Metrics")
for metric, value in all_metrics.items():
    if isinstance(value, float):
        print(f"{metric}: {value:.4f}")
    else:
        print(f"{metric}: {value}")

# ----------------------------
# 10) Save artifacts
# ----------------------------
metrics_path = REPORT_ROOT / "content_based_metrics.json"
summary_path = REPORT_ROOT / "content_based_run_summary.json"
bundle_path = ARTIFACT_ROOT / "content_based_bundle.pkl"

run_summary = {
    "input_files": {
```

```
        "interactions": str(INTERACTIONS_PATH),
        "products": str(PRODUCTS_PATH),
    },
    "product_id_column": product_id_col,
    "text_columns_used": available_text_cols,
    "train_shape": list(train_df.shape),
    "test_shape": list(test_df.shape),
    "tfidf_shape": list(tfidf_matrix.shape),
    "sample_user": str(sample_user),
    "sample_top5_recommendations": [str(x) for x in sample_recs],
    "metrics": all_metrics,
}

with open(metrics_path, "w", encoding="utf-8") as f:
    json.dump(all_metrics, f, indent=2)

with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(run_summary, f, indent=2)

with open(bundle_path, "wb") as f:
    pickle.dump(
        {
            "vectorizer": vectorizer,
            "tfidf_matrix": tfidf_matrix,
            "item_to_idx": item_to_idx,
            "idx_to_item": idx_to_item,
            "user_history": user_history,
            "product_id_column": product_id_col,
            "text_columns_used": available_text_cols,
        },
        f,
    )

# ----------------------------
# 11) MLflow logging
# ----------------------------
with mlflow.start_run(run_name="content_based_baseline_fixed_overlap") as run:
    mlflow.log_param("model_type", "content_based_filtering")
    mlflow.log_param("feature_method", "tfidf_text_plus_optional_price_bucket")
    mlflow.log_param("interactions_file", str(INTERACTIONS_PATH))
    mlflow.log_param("products_file", str(PRODUCTS_PATH))
    mlflow.log_param("product_id_column", product_id_col)
    mlflow.log_param("text_columns_used", ",".join(available_text_cols))
    mlflow.log_param("split_type", "leave_one_out_after_catalog_overlap_filter")
    mlflow.log_param("ngram_range", "1_2")
    mlflow.log_param("stop_words", "english")

    mlflow.log_param("train_rows", int(len(train_df)))
    mlflow.log_param("test_rows", int(len(test_df)))
    mlflow.log_param("train_users", int(train_df["user_id"].nunique()))
    mlflow.log_param("train_items", int(train_df["item_id"].nunique()))
    mlflow.log_param("catalog_size_used", int(len(products_cb)))
    mlflow.log_param("feature_vocab_size", int(len(vectorizer.vocabulary_)))
    mlflow.log_param("overlap_items", int(len(overlap_items)))

    mlflow.log_metrics(all_metrics)
    mlflow.log_artifact(str(metrics_path))
    mlflow.log_artifact(str(summary_path))
    mlflow.log_artifact(str(bundle_path))

    run_id = run.info.run_id

print("\nMLflow tracking URI:", tracking_uri)
print("MLflow run_id:", run_id)
print("Saved artifacts:")
print("-", metrics_path)
print("-", summary_path)
print("-", bundle_path)
```

Code: src\05-training\item-based-collaborative.ipynb

```
import json
import pickle
import time
import mlflow
import mlflow.sklearn
import numpy as np
import pandas as pd
from pathlib import Path
from scipy.sparse import csr_matrix
from sklearn.neighbors import NearestNeighbors

start = time.time()

# ----------------------------
# 0) Project / MLflow setup
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
```

```
DATA_ROOT = PROJECT_ROOT / "data"
PREP_ROOT = DATA_ROOT / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports"
ARTIFACT_ROOT = PROJECT_ROOT / "models" / "artifacts"
MLRUNS_ROOT = PROJECT_ROOT / "mlruns"

REPORT_ROOT.mkdir(parents=True, exist_ok=True)
ARTIFACT_ROOT.mkdir(parents=True, exist_ok=True)
MLRUNS_ROOT.mkdir(parents=True, exist_ok=True)

tracking_uri = f"file:///{MLRUNS_ROOT.resolve().as_posix()}"
mlflow.set_tracking_uri(tracking_uri)
mlflow.set_experiment("recomart_item_based_cf")

FILE_PATH = PREP_ROOT / "interactions_prepared.csv"

# ----------------------------
# 1) Load prepared interactions
# ----------------------------
use_cols = ["user_id", "item_id", "event_weight"]
df = pd.read_csv(FILE_PATH, usecols=use_cols)

required_cols = {"user_id", "item_id", "event_weight"}
missing = required_cols - set(df.columns)
if missing:
    raise ValueError(f"Missing required columns: {missing}")

print("Loaded:", FILE_PATH)
print("Raw shape:", df.shape)

# ----------------------------
# 2) Aggregate + reduce dataset
# ----------------------------
df = (
    df.groupby(["user_id", "item_id"], as_index=False, sort=False)["event_weight"]
      .sum()
)

MIN_USER_ITEMS = 5
MIN_ITEM_USERS = 20
MAX_USERS = 30000
MAX_ITEMS = 3000
TOP_NEIGHBORS = 100
MAX_EVAL_USERS = 10000

changed = True
while changed:
    before = len(df)

    user_counts = df.groupby("user_id")["item_id"].nunique()
    keep_users = user_counts[user_counts >= MIN_USER_ITEMS].index
    df = df[df["user_id"].isin(keep_users)]

    item_counts = df.groupby("item_id")["user_id"].nunique()
    keep_items = item_counts[item_counts >= MIN_ITEM_USERS].index
    df = df[df["item_id"].isin(keep_items)]

    changed = len(df) != before

top_users = (
    df.groupby("user_id")["item_id"]
      .nunique()
      .sort_values(ascending=False)
      .head(MAX_USERS)
      .index
)
df = df[df["user_id"].isin(top_users)]

top_items = (
    df.groupby("item_id")["user_id"]
      .nunique()
      .sort_values(ascending=False)
      .head(MAX_ITEMS)
      .index
)
df = df[df["item_id"].isin(top_items)].copy()

if df.empty:
    raise ValueError("Filtered dataset is empty. Relax thresholds.")

print("Filtered shape:", df.shape)
print("Users:", df["user_id"].nunique(), "Items:", df["item_id"].nunique())

# ----------------------------
# 3) Leave-one-out split
# ----------------------------
rng = np.random.RandomState(42)
df["_rand"] = rng.rand(len(df))
test_idx = df.groupby("user_id")["_rand"].idxmin()

test_df = df.loc[test_idx, ["user_id", "item_id", "event_weight"]].copy()
train_df = df.drop(index=test_idx).copy()
```

```
train_counts = train_df.groupby("user_id")["item_id"].size()
valid_users = train_counts[train_counts >= 1].index
train_df = train_df[train_df["user_id"].isin(valid_users)].copy()
test_df = test_df[test_df["user_id"].isin(valid_users)].copy()

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)

# ----------------------------
# 4) Build sparse matrix
# ----------------------------
user_ids = train_df["user_id"].unique()
item_ids = train_df["item_id"].unique()

user_to_idx = {u: i for i, u in enumerate(user_ids)}
item_to_idx = {it: i for i, it in enumerate(item_ids)}
idx_to_item = np.array(item_ids)

test_df = test_df[
    test_df["user_id"].isin(user_to_idx) & test_df["item_id"].isin(item_to_idx)
].copy()

rows = train_df["user_id"].map(user_to_idx).to_numpy()
cols = train_df["item_id"].map(item_to_idx).to_numpy()
vals = train_df["event_weight"].astype(np.float32).to_numpy()

user_item = csr_matrix(
    (vals, (rows, cols)),
    shape=(len(user_to_idx), len(item_to_idx)),
    dtype=np.float32
).tocsr()

print("User-item matrix:", user_item.shape)

# ----------------------------
# 5) Train fast item neighbors
# ----------------------------
item_user = user_item.T.tocsr()

nn = NearestNeighbors(
    metric="cosine",
    algorithm="brute",
    n_neighbors=min(TOP_NEIGHBORS + 1, item_user.shape[0]),
    n_jobs=-1
)
nn.fit(item_user)

distances, neighbors = nn.kneighbors(item_user, return_distance=True)
sims = 1.0 - distances

print("Neighbor index shape:", neighbors.shape)

# ----------------------------
# 6) Cache seen items once
# ----------------------------
user_seen_idx = {}
user_seen_vals = {}

for user_id, uidx in user_to_idx.items():
    start_ptr = user_item.indptr[uidx]
    end_ptr = user_item.indptr[uidx + 1]
    seen_idx = user_item.indices[start_ptr:end_ptr]
    seen_vals = user_item.data[start_ptr:end_ptr]
    user_seen_idx[user_id] = seen_idx
    user_seen_vals[user_id] = seen_vals

# ----------------------------
# 7) Recommend
# ----------------------------
def recommend_items(user_id, k=10):
    if user_id not in user_to_idx:
        return []

    seen_idx = user_seen_idx[user_id]
    seen_vals = user_seen_vals[user_id]

    scores = np.zeros(len(item_to_idx), dtype=np.float32)

    for item_idx, weight in zip(seen_idx, seen_vals):
        nbrs = neighbors[item_idx]
        nbr_sims = sims[item_idx]

        mask = nbrs != item_idx
        nbrs = nbrs[mask]
        nbr_sims = nbr_sims[mask]

        scores[nbrs] += weight * nbr_sims

    scores[seen_idx] = -np.inf

    top_n = min(k, len(scores))
    top_idx = np.argpartition(scores, -top_n)[-top_n:]
    top_idx = top_idx[np.argsort(scores[top_idx])[::-1]]
```

```
    top_idx = top_idx[np.isfinite(scores[top_idx])]

    return idx_to_item[top_idx].tolist()

sample_user = train_df["user_id"].iloc[0]
sample_recs = recommend_items(sample_user, k=5)
print("Sample user:", sample_user)
print("Top-5 recs:", sample_recs)

# ----------------------------
# 8) Evaluate
# ----------------------------
def precision_recall_ndcg_at_k(test_df, k=10, max_eval_users=10000):
    user_truth = test_df.groupby("user_id")["item_id"].apply(set).to_dict()
    eval_users = list(user_truth.keys())[:max_eval_users]

    precisions, recalls, ndcgs = [], [], []

    for user_id in eval_users:
        recs = recommend_items(user_id, k=k)
        if not recs:
            continue

        true_items = user_truth[user_id]
        hits = np.array([1 if item in true_items else 0 for item in recs], dtype=np.float32)

        precision = float(hits.sum() / k)
        recall = float(hits.sum() / len(true_items))

        discounts = 1.0 / np.log2(np.arange(2, len(hits) + 2))
        dcg = float((hits * discounts).sum())

        ideal_len = min(len(true_items), k)
        idcg = float((np.ones(ideal_len) / np.log2(np.arange(2, ideal_len + 2))).sum())
        ndcg = dcg / idcg if idcg > 0 else 0.0

        precisions.append(precision)
        recalls.append(recall)
        ndcgs.append(ndcg)

    return {
        f"precision_at_{k}": float(np.mean(precisions)) if precisions else 0.0,
        f"recall_at_{k}": float(np.mean(recalls)) if recalls else 0.0,
        f"ndcg_at_{k}": float(np.mean(ndcgs)) if ndcgs else 0.0,
        f"evaluated_users_at_{k}": int(len(precisions)),
    }

metrics_5 = precision_recall_ndcg_at_k(test_df, k=5, max_eval_users=MAX_EVAL_USERS)
metrics_10 = precision_recall_ndcg_at_k(test_df, k=10, max_eval_users=MAX_EVAL_USERS)

runtime_minutes = (time.time() - start) / 60.0

all_metrics = {
    **metrics_5,
    **metrics_10,
    "runtime_minutes": float(runtime_minutes),
}

print("\nMetrics")
for metric, value in all_metrics.items():
    print(f"{metric}: {value:.4f}" if isinstance(value, float) else f"{metric}: {value}")

# ----------------------------
# 9) Save artifacts
# ----------------------------
metrics_path = REPORT_ROOT / "item_cf_metrics.json"
summary_path = REPORT_ROOT / "item_cf_run_summary.json"
bundle_path = ARTIFACT_ROOT / "item_cf_bundle.pkl"

run_summary = {
    "input_file": str(FILE_PATH),
    "train_shape": list(train_df.shape),
    "test_shape": list(test_df.shape),
    "user_item_shape": list(user_item.shape),
    "sample_user": int(sample_user),
    "sample_top5_recommendations": [int(x) for x in sample_recs],
    "metrics": all_metrics,
}

with open(metrics_path, "w", encoding="utf-8") as f:
    json.dump(all_metrics, f, indent=2)

with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(run_summary, f, indent=2)

with open(bundle_path, "wb") as f:
    pickle.dump(
        {
            "nn_model": nn,
            "user_to_idx": user_to_idx,
            "item_to_idx": item_to_idx,
            "idx_to_item": idx_to_item,
            "neighbors": neighbors,
```

```
            "sims": sims,
            "user_seen_idx": user_seen_idx,
            "user_seen_vals": user_seen_vals,
        },
        f,
    )

# ----------------------------
# 10) MLflow logging
# ----------------------------
with mlflow.start_run(run_name="item_based_cf_baseline") as run:
    mlflow.log_param("model_type", "item_based_collaborative_filtering")
    mlflow.log_param("algorithm", "item_knn_cosine")
    mlflow.log_param("input_file", str(FILE_PATH))
    mlflow.log_param("split_type", "random_leave_one_out_after_aggregation")
    mlflow.log_param("min_user_items", MIN_USER_ITEMS)
    mlflow.log_param("min_item_users", MIN_ITEM_USERS)
    mlflow.log_param("max_users", MAX_USERS)
    mlflow.log_param("max_items", MAX_ITEMS)
    mlflow.log_param("top_neighbors", TOP_NEIGHBORS)
    mlflow.log_param("max_eval_users", MAX_EVAL_USERS)

    mlflow.log_param("train_rows", int(len(train_df)))
    mlflow.log_param("test_rows", int(len(test_df)))
    mlflow.log_param("train_users", int(train_df["user_id"].nunique()))
    mlflow.log_param("train_items", int(train_df["item_id"].nunique()))

    mlflow.log_metrics(all_metrics)

    mlflow.log_artifact(str(metrics_path))
    mlflow.log_artifact(str(summary_path))
    mlflow.log_artifact(str(bundle_path))

    mlflow.sklearn.log_model(nn, artifact_path="item_knn_model")

    run_id = run.info.run_id

print("\nMLflow tracking URI:", tracking_uri)
print("MLflow run_id:", run_id)
print("Saved artifacts:")
print("-", metrics_path)
print("-", summary_path)
print("-", bundle_path)
```

Code: src\05-training\svd.ipynb

```
import json
import pickle
import time
import mlflow
import mlflow.sklearn
import numpy as np
import pandas as pd
from pathlib import Path
from scipy.sparse import csr_matrix
from sklearn.decomposition import TruncatedSVD

start = time.time()

# ----------------------------
# 0) Project / MLflow setup
# ----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_ROOT = PROJECT_ROOT / "data"
PREP_ROOT = DATA_ROOT / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports"
ARTIFACT_ROOT = PROJECT_ROOT / "models" / "artifacts"
MLRUNS_ROOT = PROJECT_ROOT / "mlruns"

REPORT_ROOT.mkdir(parents=True, exist_ok=True)
ARTIFACT_ROOT.mkdir(parents=True, exist_ok=True)
MLRUNS_ROOT.mkdir(parents=True, exist_ok=True)

tracking_uri = f"file:///{MLRUNS_ROOT.resolve().as_posix()}"
mlflow.set_tracking_uri(tracking_uri)
mlflow.set_experiment("recomart_svd_baseline")

FILE_PATH = PREP_ROOT / "interactions_prepared.csv"

# ----------------------------
# 1) Load prepared interactions
# ----------------------------
use_cols = ["user_id", "item_id", "event_weight"]
df = pd.read_csv(FILE_PATH, usecols=use_cols)

required_cols = {"user_id", "item_id", "event_weight"}
missing = required_cols - set(df.columns)
```

```
if missing:
    raise ValueError(f"Missing required columns: {missing}")

print("Loaded file:", FILE_PATH)
print("Raw shape:", df.shape)

# ----------------------------
# 2) Aggregate + reduce dataset
# ----------------------------
df = (
    df.groupby(["user_id", "item_id"], as_index=False, sort=False)["event_weight"]
      .sum()
)

MIN_USER_ITEMS = 5
MIN_ITEM_USERS = 20
MAX_USERS = 50000
MAX_ITEMS = 5000

changed = True
while changed:
    before = len(df)

    user_counts = df.groupby("user_id")["item_id"].nunique()
    keep_users = user_counts[user_counts >= MIN_USER_ITEMS].index
    df = df[df["user_id"].isin(keep_users)]

    item_counts = df.groupby("item_id")["user_id"].nunique()
    keep_items = item_counts[item_counts >= MIN_ITEM_USERS].index
    df = df[df["item_id"].isin(keep_items)]

    changed = len(df) != before

top_users = (
    df.groupby("user_id")["item_id"]
    .nunique()
    .sort_values(ascending=False)
    .head(MAX_USERS)
    .index
)
df = df[df["user_id"].isin(top_users)]

top_items = (
    df.groupby("item_id")["user_id"]
    .nunique()
    .sort_values(ascending=False)
    .head(MAX_ITEMS)
    .index
)
df = df[df["item_id"].isin(top_items)].copy()

if df.empty:
    raise ValueError("Filtered dataset is empty. Relax thresholds.")

print("Filtered shape:", df.shape)
print("Unique users:", df["user_id"].nunique())
print("Unique items:", df["item_id"].nunique())

# ----------------------------
# 3) Leave-one-out split
# ----------------------------
rand = np.random.RandomState(42)
df["_rand"] = rand.rand(len(df))
test_idx = df.groupby("user_id")["_rand"].idxmin()

test_df = df.loc[test_idx, ["user_id", "item_id", "event_weight"]].copy()
train_df = df.drop(index=test_idx).copy()

train_counts = train_df.groupby("user_id")["item_id"].size()
valid_users = train_counts[train_counts >= 1].index
train_df = train_df[train_df["user_id"].isin(valid_users)].copy()
test_df = test_df[test_df["user_id"].isin(valid_users)].copy()

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)

# ----------------------------
# 4) Build train matrix
# ----------------------------
user_ids = train_df["user_id"].unique()
item_ids = train_df["item_id"].unique()

user_to_idx = {u: i for i, u in enumerate(user_ids)}
item_to_idx = {it: i for i, it in enumerate(item_ids)}
idx_to_item = np.array(item_ids)

test_df = test_df[
    test_df["user_id"].isin(user_to_idx) & test_df["item_id"].isin(item_to_idx)
].copy()

rows = train_df["user_id"].map(user_to_idx).to_numpy()
cols = train_df["item_id"].map(item_to_idx).to_numpy()
vals = train_df["event_weight"].astype(np.float32).to_numpy()
```

```
user_item = csr_matrix(
    (vals, (rows, cols)),
    shape=(len(user_to_idx), len(item_to_idx)),
    dtype=np.float32
)

print("User-item matrix shape:", user_item.shape)

seen_by_user = train_df.groupby("user_id")["item_id"].apply(list).to_dict()
seen_idx_by_user = {
    u: np.array([item_to_idx[i] for i in items if i in item_to_idx], dtype=np.int32)
    for u, items in seen_by_user.items()
}

# ----------------------------
# 5) Train SVD
# ----------------------------
min_dim = min(user_item.shape)
n_components = min(20, max(2, min_dim - 1))

svd = TruncatedSVD(n_components=n_components, random_state=42)
user_factors = svd.fit_transform(user_item).astype(np.float32)
item_factors = svd.components_.astype(np.float32)

print("Chosen n_components:", n_components)
print("Explained variance ratio sum:", round(svd.explained_variance_ratio_.sum(), 4))

# ----------------------------
# 6) Recommend fast
# ----------------------------
def recommend_svd(user_id, k=10):
    if user_id not in user_to_idx:
        return []

    uidx = user_to_idx[user_id]
    scores = user_factors[uidx] @ item_factors

    seen = seen_idx_by_user.get(user_id)
    if seen is not None and len(seen) > 0:
        scores[seen] = -np.inf

    top_n = min(k, len(scores))
    top_idx = np.argpartition(scores, -top_n)[-top_n:]
    top_idx = top_idx[np.argsort(scores[top_idx])[::-1]]

    return idx_to_item[top_idx].tolist()

# ----------------------------
# 7) Ranking metrics
# ----------------------------
def evaluate_ranking_metrics(test_df, recommend_fn, k=10, max_eval_users=10000):
    user_truth = test_df.groupby("user_id")["item_id"].apply(set).to_dict()
    eval_users = list(user_truth.keys())[:max_eval_users]

    precisions, recalls, ndcgs = [], [], []

    for user_id in eval_users:
        true_items = user_truth[user_id]
        recs = recommend_fn(user_id, k=k)
        if not recs:
            continue

        hits = np.array([1 if item in true_items else 0 for item in recs], dtype=np.float32)
        hit_count = hits.sum()

        precision = hit_count / k
        recall = hit_count / len(true_items)

        discounts = 1.0 / np.log2(np.arange(2, len(hits) + 2))
        dcg = float((hits * discounts).sum())

        ideal_len = min(len(true_items), k)
        idcg = float((np.ones(ideal_len) / np.log2(np.arange(2, ideal_len + 2))).sum())
        ndcg = dcg / idcg if idcg > 0 else 0.0

        precisions.append(precision)
        recalls.append(recall)
        ndcgs.append(ndcg)

    return {
        f"precision_at_{k}": float(np.mean(precisions)) if precisions else 0.0,
        f"recall_at_{k}": float(np.mean(recalls)) if recalls else 0.0,
        f"ndcg_at_{k}": float(np.mean(ndcgs)) if ndcgs else 0.0,
        f"evaluated_users_at_{k}": int(len(precisions)),
    }

metrics_5 = evaluate_ranking_metrics(test_df, recommend_svd, k=5, max_eval_users=10000)
metrics_10 = evaluate_ranking_metrics(test_df, recommend_svd, k=10, max_eval_users=10000)

runtime_minutes = (time.time() - start) / 60.0

all_metrics = {
    **metrics_5,
```

```
    **metrics_10,
    "explained_variance_ratio_sum": float(svd.explained_variance_ratio_.sum()),
    "runtime_minutes": float(runtime_minutes),
}

print("\nSVD Metrics")
for metric, value in all_metrics.items():
    print(f"{metric}: {value:.4f}" if isinstance(value, float) else f"{metric}: {value}")

# ----------------------------
# 8) Save artifacts + log to MLflow
# ----------------------------
sample_user = train_df["user_id"].iloc[0]
sample_recs = recommend_svd(sample_user, k=5)

run_payload = {
    "file_path": str(FILE_PATH),
    "train_shape": list(train_df.shape),
    "test_shape": list(test_df.shape),
    "user_item_shape": list(user_item.shape),
    "sample_user": int(sample_user),
    "sample_top5_recommendations": [int(x) for x in sample_recs],
    "metrics": all_metrics,
}

metrics_path = REPORT_ROOT / "svd_metrics.json"
summary_path = REPORT_ROOT / "svd_run_summary.json"
model_path = ARTIFACT_ROOT / "svd_model.pkl"

with open(metrics_path, "w", encoding="utf-8") as f:
    json.dump(all_metrics, f, indent=2)

with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(run_payload, f, indent=2)

with open(model_path, "wb") as f:
    pickle.dump(
        {
            "svd": svd,
            "user_to_idx": user_to_idx,
            "item_to_idx": item_to_idx,
            "idx_to_item": idx_to_item,
            "user_factors": user_factors,
            "item_factors": item_factors,
            "seen_idx_by_user": seen_idx_by_user,
        },
        f,
    )

with mlflow.start_run(run_name="svd_recommender_baseline") as run:
    mlflow.log_param("model_type", "collaborative_filtering_svd")
    mlflow.log_param("input_file", str(FILE_PATH))
    mlflow.log_param("min_user_items", MIN_USER_ITEMS)
    mlflow.log_param("min_item_users", MIN_ITEM_USERS)
    mlflow.log_param("max_users", MAX_USERS)
    mlflow.log_param("max_items", MAX_ITEMS)
    mlflow.log_param("n_components", n_components)
    mlflow.log_param("split_type", "random_leave_one_out_after_aggregation")

    mlflow.log_param("train_rows", int(len(train_df)))
    mlflow.log_param("test_rows", int(len(test_df)))
    mlflow.log_param("train_users", int(train_df["user_id"].nunique()))
    mlflow.log_param("train_items", int(train_df["item_id"].nunique()))

    mlflow.log_metrics(all_metrics)

    mlflow.log_artifact(str(metrics_path))
    mlflow.log_artifact(str(summary_path))
    mlflow.log_artifact(str(model_path))

    mlflow.sklearn.log_model(svd, artifact_path="svd_sklearn_model")

    run_id = run.info.run_id

print("\nMLflow tracking URI:", tracking_uri)
print("MLflow run_id:", run_id)
print("Saved artifacts:")
print("-", metrics_path)
print("-", summary_path)
print("-", model_path)
```

Code: src\06-orchestration\pipeline_orchestration_prefect.ipynb

```
# Cell 1 — Setup paths and folders
from pathlib import Path
import os
import re
import json
from datetime import datetime
```

```
PROJECT_ROOT = Path(r"C:\Users\barath\recomart-pipeline")
RUN_TS = datetime.now().strftime("%Y%m%d_%H%M%S")

LOG_DIR = PROJECT_ROOT / "logs" / "orchestration"
MONITOR_DIR = PROJECT_ROOT / "logs" / "monitoring"
RUN_DIR = PROJECT_ROOT / "runs" / "prefect" / RUN_TS
EXEC_NOTEBOOK_DIR = RUN_DIR / "executed_notebooks"

for p in [LOG_DIR, MONITOR_DIR, RUN_DIR, EXEC_NOTEBOOK_DIR]:
    p.mkdir(parents=True, exist_ok=True)

os.chdir(PROJECT_ROOT)

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RUN_TS:", RUN_TS)
print("Working directory:", Path.cwd())


# ---- next code cell ----

# Cell 2 — Install Prefect and dependencies

import sys
import subprocess
import importlib
import os

def ensure_package(import_name, pip_name=None):
    pip_name = pip_name or import_name
    try:
        importlib.import_module(import_name)
        print(f"[OK] {pip_name} already available")
    except ImportError:
        print(f"[INSTALL] {pip_name}")
        subprocess.check_call([sys.executable, "-m", "pip", "install", pip_name])

packages = [
    ("prefect", "prefect"),
    ("papermill", "papermill"),
    ("nbformat", "nbformat"),
    ("nbclient", "nbclient"),
    ("ipykernel", "ipykernel"),
    ("requests", "requests"),
]

for import_name, pip_name in packages:
    ensure_package(import_name, pip_name)

os.environ["PREFECT_LOGGING_LEVEL"] = "INFO"
os.environ["PYTHONIOENCODING"] = "utf-8"

print("Dependency check complete.")



# ---- next code cell ----

#Cell 3 — Start local Prefect server/UI
# Code Generated by Sidekick is for learning and experimentation purposes only.
import socket
import time
import os
import subprocess
import sys
import webbrowser
import requests

PREFECT_UI_URL = "http://127.0.0.1:4200"
PREFECT_API_URL = "http://127.0.0.1:4200/api"
PREFECT_SERVER_LOG = LOG_DIR / f"prefect_server_{RUN_TS}.log"

def is_port_open(host="127.0.0.1", port=4200):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(1)
        return sock.connect_ex((host, port)) == 0

def wait_for_prefect_api(api_url=PREFECT_API_URL, timeout=120):
    start = time.time()
    health_url = f"{api_url}/health"
    while time.time() - start < timeout:
        try:
            response = requests.get(health_url, timeout=3)
            if response.status_code == 200:
                return True
        except Exception:
            pass
        time.sleep(2)
    return False

server_process = None

if is_port_open():
    print("Prefect server already running on port 4200.")
```

```
else:
    log_file = open(PREFECT_SERVER_LOG, "w", encoding="utf-8")
    server_process = subprocess.Popen(
        [sys.executable, "-m", "prefect", "server", "start"],
        cwd=str(PROJECT_ROOT),
        stdout=log_file,
        stderr=subprocess.STDOUT,
        env={**os.environ, "PREFECT_API_URL": PREFECT_API_URL}
    )
    print("Started Prefect server process:", server_process.pid)
    print("Server log:", PREFECT_SERVER_LOG)

ready = wait_for_prefect_api()
if not ready:
    raise RuntimeError("Prefect server did not become ready within timeout.")

os.environ["PREFECT_API_URL"] = PREFECT_API_URL
print("Prefect UI:", PREFECT_UI_URL)
print("Prefect API:", PREFECT_API_URL)

try:
    webbrowser.open(PREFECT_UI_URL)
except Exception:
    pass


# ---- next code cell ----

# Cell 4 — Define your notebook paths
NOTEBOOK_PATHS = {
    "ingest_dummyjson": PROJECT_ROOT / "src" / "01-ingestion" / "dummyjson_ingestion.ipynb",
    "ingest_retailrocket": PROJECT_ROOT / "src" / "01-ingestion" / "retailrocket_ingestion.ipynb",
    "validate_raw": PROJECT_ROOT / "src" / "02-validation" / "run_validation.ipynb",
    "eda_prep": PROJECT_ROOT / "src" / "03-eda-prep" / "eda_prep-copy.ipynb",
    "materialize_features": PROJECT_ROOT / "src" / "04-features" / "materialize_features.ipynb",
    "registry_loader": PROJECT_ROOT / "src" / "04-features" / "registry_loader.ipynb",
    "train_item_based": PROJECT_ROOT / "src" / "05-training" / "item-based-collaborative.ipynb",
    "train_svd": PROJECT_ROOT / "src" / "05-training" / "svd.ipynb",
    "train_content_based": PROJECT_ROOT / "src" / "05-training" / "content-based-training.ipynb",
    "compare_models": PROJECT_ROOT / "src" / "05-training" / "comparision.ipynb",
}

for name, path in NOTEBOOK_PATHS.items():
    print(f"{name:22} -> {'FOUND' if path.exists() else 'MISSING'} | {path}")


# ---- next code cell ----

# Cell 5 — Logging and notebook execution helpers
import logging

def _safe_name(text: str) -> str:
    return re.sub(r"[^A-Za-z0-9]+", "_", text).strip("_").lower()

def get_logger(name: str):
    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)

    if not logger.handlers:
        log_file = LOG_DIR / f"{_safe_name(name)}_{RUN_TS}.log"
        fh = logging.FileHandler(log_file, encoding="utf-8")
        sh = logging.StreamHandler(sys.stdout)

        formatter = logging.Formatter("%(asctime)s | %(levelname)s | %(name)s | %(message)s")
        fh.setFormatter(formatter)
        sh.setFormatter(formatter)

        logger.addHandler(fh)
        logger.addHandler(sh)

    return logger

def write_status(stage: str, status: str, notebook_path: Path = None, error: str = None, output_notebook: Path = None):
    payload = {
        "run_ts": RUN_TS,
        "stage": stage,
        "status": status,
        "notebook_path": str(notebook_path) if notebook_path else None,
        "output_notebook": str(output_notebook) if output_notebook else None,
        "error": error,
        "event_ts": datetime.now().isoformat()
    }
    out_path = MONITOR_DIR / f"{_safe_name(stage)}_{status}_{RUN_TS}.json"
    out_path.write_text(json.dumps(payload, indent=2), encoding="utf-8")
    return out_path

def execute_notebook(stage: str, notebook_path: Path):
    logger = get_logger(stage)

    if not notebook_path.exists():
        write_status(stage, "failed", notebook_path=notebook_path, error="Notebook file not found")
        raise FileNotFoundError(f"Notebook not found: {notebook_path}")
```

```
    output_dir = EXEC_NOTEBOOK_DIR / _safe_name(stage)
    output_dir.mkdir(parents=True, exist_ok=True)
    output_notebook = output_dir / f"{notebook_path.stem}_executed.ipynb"

    cmd = [
        sys.executable,
        "-m",
        "papermill",
        str(notebook_path),
        str(output_notebook),
    ]

    logger.info("Starting stage: %s", stage)
    logger.info("Command: %s", " ".join(cmd))
    write_status(stage, "started", notebook_path=notebook_path, output_notebook=output_notebook)

    result = subprocess.run(
        cmd,
        cwd=str(PROJECT_ROOT),
        capture_output=True,
        text=True,
        env={**os.environ, "PREFECT_API_URL": PREFECT_API_URL}
    )

    if result.stdout:
        logger.info("STDOUT:\n%s", result.stdout)

    if result.stderr:
        logger.warning("STDERR:\n%s", result.stderr)

    if result.returncode != 0:
        write_status(
            stage,
            "failed",
            notebook_path=notebook_path,
            output_notebook=output_notebook,
            error=result.stderr[-3000:] if result.stderr else f"Return code {result.returncode}"
        )
        raise RuntimeError(f"{stage} failed with return code {result.returncode}")

    write_status(stage, "success", notebook_path=notebook_path, output_notebook=output_notebook)
    logger.info("Completed stage: %s", stage)
    return str(output_notebook)


# ---- next code cell ----

# Cell 6 — Print DAG graph
from IPython.display import HTML, display

dag_html = """
<div style="font-family: Arial, sans-serif; line-height: 1.6;">
  <h3>RecoMart Prefect Orchestration DAG</h3>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#86BC25;color:white;border-radius:8px;">DummyJSON Ingestion</spa
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-radius:8px;">Validation</span>
  </div>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#43B02A;color:white;border-radius:8px;">RetailRocket Ingestion<
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-radius:8px;">Validation</span>
  </div>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#0076A8;color:white;border-radius:8px;">Validation</span>
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#1D4F91;color:white;border-radius:8px;">EDA / Preparation</span
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#005587;color:white;border-radius:8px;">Materialize Features</sp
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#046E38;color:white;border-radius:8px;">Registry Loader</span>
  </div>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#007CB0;color:white;border-radius:8px;">Item-Based Training</spa
    <span style="font-size:22px;margin:0 8px;">■</span>
    <span style="display:inline-block;padding:10px 14px;background:#62B5E5;color:white;border-radius:8px;">Content-Based Training<
  </div>

  <div style="margin: 12px 0;">
    <span style="display:inline-block;padding:10px 14px;background:#00A9E0;color:white;border-radius:8px;">SVD Training</span>
    <span style="font-size:22px;margin:0 8px;">■</span>
    <span style="display:inline-block;padding:10px 14px;background:#62B5E5;color:white;border-radius:8px;">Content-Based Training<
    <span style="font-size:22px;margin:0 8px;">→</span>
    <span style="display:inline-block;padding:10px 14px;background:#63666A;color:white;border-radius:8px;">Comparison</span>
  </div>
</div>
"""

display(HTML(dag_html))
```

```
# ---- next code cell ----

# Cell 7 — Define Prefect tasks and flow
from prefect import flow, task, get_run_logger

@task(name="ingest_dummyjson", retries=2, retry_delay_seconds=20)
def ingest_dummyjson():
    return execute_notebook("ingest_dummyjson", NOTEBOOK_PATHS["ingest_dummyjson"])

@task(name="ingest_retailrocket", retries=2, retry_delay_seconds=20)
def ingest_retailrocket():
    return execute_notebook("ingest_retailrocket", NOTEBOOK_PATHS["ingest_retailrocket"])

@task(name="validate_raw", retries=1, retry_delay_seconds=10)
def validate_raw():
    return execute_notebook("validate_raw", NOTEBOOK_PATHS["validate_raw"])

@task(name="eda_prep", retries=1, retry_delay_seconds=10)
def eda_prep():
    return execute_notebook("eda_prep", NOTEBOOK_PATHS["eda_prep"])

@task(name="materialize_features", retries=1, retry_delay_seconds=10)
def materialize_features():
    return execute_notebook("materialize_features", NOTEBOOK_PATHS["materialize_features"])

@task(name="registry_loader", retries=1, retry_delay_seconds=10)
def registry_loader():
    return execute_notebook("registry_loader", NOTEBOOK_PATHS["registry_loader"])

@task(name="train_item_based", retries=0)
def train_item_based():
    return execute_notebook("train_item_based", NOTEBOOK_PATHS["train_item_based"])

@task(name="train_svd", retries=0)
def train_svd():
    return execute_notebook("train_svd", NOTEBOOK_PATHS["train_svd"])

@task(name="train_content_based", retries=0)
def train_content_based():
    return execute_notebook("train_content_based", NOTEBOOK_PATHS["train_content_based"])

@task(name="compare_models", retries=0)
def compare_models():
    return execute_notebook("compare_models", NOTEBOOK_PATHS["compare_models"])

@flow(name="recomart_assignment_prefect_flow", log_prints=True)
def recomart_assignment_prefect_flow():
    logger = get_run_logger()
    logger.info("Pipeline started")

    pipeline_status = {
        "run_ts": RUN_TS,
        "status": "started",
        "event_ts": datetime.now().isoformat()
    }
    (MONITOR_DIR / f"pipeline_started_{RUN_TS}.json").write_text(
        json.dumps(pipeline_status, indent=2), encoding="utf-8"
    )

    try:
        dummy_future = ingest_dummyjson.submit()
        retail_future = ingest_retailrocket.submit()

        dummy_future.result()
        retail_future.result()

        validate_raw.submit().result()
        eda_prep.submit().result()

        materialize_features.submit().result()
        registry_loader.submit().result()

        item_future = train_item_based.submit()
        svd_future = train_svd.submit()

        item_future.result()
        svd_future.result()

        train_content_based.submit().result()
        compare_models.submit().result()

        pipeline_status["status"] = "success"
        pipeline_status["event_ts"] = datetime.now().isoformat()
        (MONITOR_DIR / f"pipeline_success_{RUN_TS}.json").write_text(
            json.dumps(pipeline_status, indent=2), encoding="utf-8"
        )

        logger.info("Pipeline completed successfully")

    except Exception as exc:
        pipeline_status["status"] = "failed"
        pipeline_status["error"] = str(exc)
        pipeline_status["event_ts"] = datetime.now().isoformat()
```

```
        (MONITOR_DIR / f"pipeline_failed_{RUN_TS}.json").write_text(
            json.dumps(pipeline_status, indent=2), encoding="utf-8"
        )
        logger.exception("Pipeline failed: %s", exc)
        raise


# ---- next code cell ----

# Cell 8 — Validate all notebook files exist
missing = [str(path) for path in NOTEBOOK_PATHS.values() if not path.exists()]

if missing:
    print("Missing notebook files:")
    for m in missing:
        print("-", m)
    raise FileNotFoundError("Fix missing notebook paths before running the flow.")
else:
    print("All notebook paths found.")


# ---- next code cell ----

#Cell 9 — Run the full orchestration flow
recomart_assignment_prefect_flow()


# ---- next code cell ----

# Cell 10 — Show logs, monitoring, and executed notebooks
print("Monitoring files:")
for p in sorted(MONITOR_DIR.glob(f"*{RUN_TS}*.json")):
    print("-", p)

print("\nExecuted notebooks:")
for p in sorted(EXEC_NOTEBOOK_DIR.rglob("*.ipynb")):
    print("-", p)

print("\nTask logs:")
for p in sorted(LOG_DIR.glob(f"*{RUN_TS}.log")):
    print("-", p)


# ---- next code cell ----

# Cell 11 — Optional cleanup: stop Prefect server started by notebook
if "server_process" in globals() and server_process is not None:
    server_process.terminate()
    print(f"Stopped Prefect server process: {server_process.pid}")
else:
    print("No notebook-started Prefect server process found.")
```

Code: src\07-submission\final_submission.ipynb

```
from __future__ import annotations

import json
import re
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple
from xml.sax.saxutils import escape

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.platypus import (
    PageBreak,
    Paragraph,
    Preformatted,
    SimpleDocTemplate,
    Spacer,
    Table,
    TableStyle,
)

# ============================================================
# RecoMart - Personalized Final Submission PDF Generator
# ============================================================

PROJECT_ROOT = Path(r"C:\Users\barath\recomart-pipeline")
RAW_ROOT = PROJECT_ROOT / "data" / "raw"
BRONZE_ROOT = PROJECT_ROOT / "data" / "bronze"
REMEDIATED_RAW_ROOT = PROJECT_ROOT / "data" / "remediated" / "raw"
REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / "data" / "remediated" / "bronze"
REPORT_ROOT = PROJECT_ROOT / "reports" / "validation"
LOG_ROOT = PROJECT_ROOT / "logs"
RUNS_ROOT = PROJECT_ROOT / "runs" / "prefect"
MLRUNS_ROOT = PROJECT_ROOT / "mlruns"
```

```
FEATURE_STORE_ROOT = PROJECT_ROOT / "feature_store"
SRC_ROOT = PROJECT_ROOT / "src"

OUTPUT_DIR = PROJECT_ROOT / "reports" / "submission"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_PDF = OUTPUT_DIR / "RecoMart_Final_Submission_Barath.pdf"

# Exact validation evidence observed in uploaded files
VALIDATION_RUN_ID = "20260428T144031Z"
VALIDATION_INITIAL_STATUS = "FAIL"
VALIDATION_FINAL_STATUS = "PASS"

DISCOVERED_FILES = {
    "events_csv": RAW_ROOT / "retailrocket" / "load_date=2026-04-28" / "load_hour=14" / "events.csv",
    "category_tree_csv": RAW_ROOT / "retailrocket" / "load_date=2026-04-26" / "load_hour=19" / "category_tree.csv",
    "item_properties_part1_csv": RAW_ROOT / "retailrocket" / "load_date=2026-04-28" / "load_hour=14" / "item_properties_part1.csv",
    "item_properties_part2_csv": RAW_ROOT / "retailrocket" / "load_date=2026-04-28" / "load_hour=14" / "item_properties_part2.csv",
    "products_raw_json": RAW_ROOT / "dummyjson" / "load_date=2026-04-28" / "load_hour=14" / "products_raw.json",
    "categories_raw_json": RAW_ROOT / "dummyjson" / "load_date=2026-04-28" / "load_hour=14" / "categories_raw.json",
    "products_parquet": BRONZE_ROOT / "dummyjson" / "products.parquet",
    "categories_parquet": BRONZE_ROOT / "dummyjson" / "categories.parquet",
}

VALIDATION_ARTIFACTS = {
    "initial_issues": REPORT_ROOT / f"validation_issues_initial_{VALIDATION_RUN_ID}.csv",
    "initial_summary": REPORT_ROOT / f"validation_summary_initial_{VALIDATION_RUN_ID}.csv",
    "revalidation_issues": REPORT_ROOT / f"validation_issues_revalidated_{VALIDATION_RUN_ID}.csv",
    "revalidation_summary": REPORT_ROOT / f"validation_summary_revalidated_{VALIDATION_RUN_ID}.csv",
    "fix_log": REPORT_ROOT / f"fix_log_{VALIDATION_RUN_ID}.csv",
    "json_report": REPORT_ROOT / f"data_quality_report_{VALIDATION_RUN_ID}.json",
    "pdf_report": REPORT_ROOT / f"data_quality_report_{VALIDATION_RUN_ID}.pdf",
    "validation_log": LOG_ROOT / f"validation_log_{VALIDATION_RUN_ID}.jsonl",
}

SUBMISSION = {
    "student_name": "Barath",
    "roll_number": "__________",
    "course_name": "Data Management for Machine Learning",
    "assignment_title": "End-to-End Data Management Pipeline for a Recommendation System",
    "client_name": "RecoMart",
    "submission_date": datetime.now().strftime("%Y-%m-%d"),
    "business_problem": (
        "RecoMart requires a scalable, maintainable, and automated data management pipeline "
        "to ingest fresh multi-source e-commerce data, validate and remediate quality issues, "
        "prepare and transform datasets, manage reusable recommendation features, and train or "
        "refresh recommendation models with orchestration and experiment tracking."
    ),
    "objectives": [
        "Ingest at least two data types: CSV-based interaction/catalog files and API/raw JSON product data.",
        "Store source data in a structured local data lake partitioned by source and timestamp.",
        "Validate data quality, produce issue logs, auto-remediate failed datasets, and revalidate.",
        "Prepare clean datasets for EDA and downstream recommendation modeling.",
        "Implement feature engineering and a custom metadata-based feature store with versioned retrieval.",
        "Track model runs, parameters, and metrics using MLflow or equivalent metadata tracking.",
        "Orchestrate the full workflow end to end using Prefect.",
    ],
    "evaluation_metrics": ["Precision@K", "Recall@K", "NDCG@K"],
}

# ---------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------
styles = getSampleStyleSheet()
styles.add(
    ParagraphStyle(
        name="SectionBody",
        parent=styles["BodyText"],
        fontSize=10,
        leading=14,
        spaceAfter=6,
    )
)
styles.add(
    ParagraphStyle(
        name="SmallCode",
        fontName="Courier",
        fontSize=7,
        leading=8,
        spaceAfter=6,
    )
)

def p(text: str, style: str = "SectionBody") -> Paragraph:
    return Paragraph(escape(text).replace("\n", "<br/>"), styles[style])

def bullets(items: List[str]) -> List[Any]:
    out: List[Any] = []
    for item in items:
        out.append(Paragraph(f"• {escape(str(item))}", styles["SectionBody"]))
    out.append(Spacer(1, 6))
    return out

def add_table(story: List[Any], title: str, rows: List[List[str]], col_widths=None) -> None:
```

```
    story.append(Paragraph(title, styles["Heading3"]))
    table = Table(rows, repeatRows=1, colWidths=col_widths)
    table.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#D9EAD3")),
                ("TEXTCOLOR", (0, 0), (-1, 0), colors.black),
                ("GRID", (0, 0), (-1, -1), 0.4, colors.grey),
                ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
                ("FONTSIZE", (0, 0), (-1, -1), 8),
                ("VALIGN", (0, 0), (-1, -1), "TOP"),
                ("LEFTPADDING", (0, 0), (-1, -1), 4),
                ("RIGHTPADDING", (0, 0), (-1, -1), 4),
                ("TOPPADDING", (0, 0), (-1, -1), 4),
                ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
            ]
        )
    )
    story.append(table)
    story.append(Spacer(1, 10))

def safe_read_text(path: Path) -> str:
    try:
        return path.read_text(encoding="utf-8", errors="ignore")
    except Exception as exc:
        return f"[Could not read {path}: {exc}]"

def parse_notebook_code(path: Path) -> str:
    try:
        payload = json.loads(path.read_text(encoding="utf-8"))
        cells = []
        for cell in payload.get("cells", []):
            if cell.get("cell_type") == "code":
                src = "".join(cell.get("source", []))
                if src.strip():
                    cells.append(src)
        return "\n\n# ---- next code cell ----\n\n".join(cells) if cells else "[No code cells found]"
    except Exception as exc:
        return f"[Could not parse notebook {path}: {exc}]"

def gather_code_files() -> List[Path]:
    allowed = {".py", ".ipynb", ".yaml", ".yml", ".md", ".txt"}
    files: List[Path] = []
    if SRC_ROOT.exists():
        for path in SRC_ROOT.rglob("*"):
            if path.is_file() and path.suffix.lower() in allowed:
                files.append(path)

    # Add feature store configs if present
    if FEATURE_STORE_ROOT.exists():
        for path in FEATURE_STORE_ROOT.rglob("*"):
            if path.is_file() and path.suffix.lower() in allowed:
                files.append(path)

    return sorted(set(files))

def latest_prefect_run() -> Dict[str, str]:
    result = {
        "flow_name": "Not found",
        "latest_run_folder": "Not found",
        "executed_notebooks_folder": "Not found",
        "status_note": "Prefect run metadata could not be inferred.",
    }

    if not RUNS_ROOT.exists():
        result["status_note"] = "runs/prefect folder not found."
        return result

    run_dirs = [p for p in RUNS_ROOT.iterdir() if p.is_dir()]
    if not run_dirs:
        result["status_note"] = "No Prefect run folders found."
        return result

    latest = sorted(run_dirs, key=lambda p: p.name)[-1]
    result["latest_run_folder"] = str(latest)

    executed = latest / "executed_notebooks"
    if executed.exists():
        result["executed_notebooks_folder"] = str(executed)
        result["flow_name"] = latest.name
        result["status_note"] = "Latest Prefect run folder discovered successfully."
    else:
        result["flow_name"] = latest.name
        result["status_note"] = "Latest Prefect run found, but executed_notebooks folder missing."

    return result

def read_mlflow_meta_yaml(meta_file: Path) -> Dict[str, str]:
    data: Dict[str, str] = {}
    text = safe_read_text(meta_file)
    for line in text.splitlines():
        if ":" in line:
            k, v = line.split(":", 1)
```

```
            data[k.strip()] = v.strip().strip("'").strip('"')
    return data

def try_parse_metric_file(metric_file: Path) -> Optional[float]:
    try:
        lines = [line.strip() for line in safe_read_text(metric_file).splitlines() if line.strip()]
        if not lines:
            return None
        last = lines[-1].split()
        return float(last[-1])
    except Exception:
        return None

def discover_mlflow_summary() -> Dict[str, Any]:
    summary = {
        "experiment_name": "Not found",
        "best_run_id": "Not found",
        "best_metric_name": "Not found",
        "best_metric_value": "Not found",
        "artifact_uri": "Not found",
        "status_note": "mlruns folder not found.",
    }

    if not MLRUNS_ROOT.exists():
        return summary

    experiment_dirs = [
        p for p in MLRUNS_ROOT.iterdir()
        if p.is_dir() and p.name != ".trash" and re.fullmatch(r"\d+", p.name)
    ]
    if not experiment_dirs:
        summary["status_note"] = "No MLflow experiment folders found."
        return summary

    # pick latest modified experiment
    experiment = sorted(experiment_dirs, key=lambda p: p.stat().st_mtime)[-1]
    exp_meta = experiment / "meta.yaml"
    if exp_meta.exists():
        exp_info = read_mlflow_meta_yaml(exp_meta)
        summary["experiment_name"] = exp_info.get("name", experiment.name)

    best_run = None
    best_metric_name = None
    best_metric_value = None
    best_artifact_uri = None

    for run_dir in experiment.iterdir():
        if not run_dir.is_dir():
            continue
        if run_dir.name in {"meta.yaml", "tags"}:
            continue

        meta_yaml = run_dir / "meta.yaml"
        metrics_dir = run_dir / "metrics"

        artifact_uri = "Not found"
        if meta_yaml.exists():
            run_info = read_mlflow_meta_yaml(meta_yaml)
            artifact_uri = run_info.get("artifact_uri", "Not found")

        if metrics_dir.exists():
            for metric_file in metrics_dir.iterdir():
                if metric_file.is_file():
                    value = try_parse_metric_file(metric_file)
                    if value is None:
                        continue
                    if best_metric_value is None or value > best_metric_value:
                        best_metric_value = value
                        best_metric_name = metric_file.name
                        best_run = run_dir.name
                        best_artifact_uri = artifact_uri

    if best_run:
        summary["best_run_id"] = best_run
        summary["best_metric_name"] = best_metric_name or "Not found"
        summary["best_metric_value"] = best_metric_value
        summary["artifact_uri"] = best_artifact_uri or "Not found"
        summary["status_note"] = "MLflow run and metric discovered from local mlruns folder."
    else:
        summary["status_note"] = "Experiment found, but no readable metric files were discovered."

    return summary

def infer_stage_folders() -> List[str]:
    if not SRC_ROOT.exists():
        return []
    return [p.name for p in sorted(SRC_ROOT.iterdir()) if p.is_dir()]

def repo_tree(root: Path, max_depth: int = 2) -> str:
    if not root.exists():
        return f"{root} [not found]"

    lines: List[str] = [root.name + "/"]
```

```
    def walk(path: Path, prefix: str, depth: int) -> None:
        if depth > max_depth:
            return
        children = sorted(path.iterdir(), key=lambda x: (x.is_file(), x.name.lower()))
        for child in children:
            lines.append(f"{prefix}{child.name}{'/' if child.is_dir() else ''}")
            if child.is_dir():
                walk(child, prefix + "    ", depth + 1)

    walk(root, "    ", 0)
    return "\n".join(lines)

def code_listing(path: Path) -> str:
    if path.suffix.lower() == ".ipynb":
        return parse_notebook_code(path)
    return safe_read_text(path)

# ------------------------------------------------------------
# Build report
# ------------------------------------------------------------
def build_story() -> List[Any]:
    prefect_info = latest_prefect_run()
    mlflow_info = discover_mlflow_summary()
    stage_folders = infer_stage_folders()
    code_files = gather_code_files()

    story: List[Any] = []

    # Title
    story.append(Paragraph("RecoMart Final Assignment Submission Report", styles["Title"]))
    story.append(Spacer(1, 12))
    story.append(p(f"Student Name: {SUBMISSION['student_name']}"))
    story.append(p(f"Roll Number: {SUBMISSION['roll_number']}"))
    story.append(p(f"Course: {SUBMISSION['course_name']}"))
    story.append(p(f"Assignment Title: {SUBMISSION['assignment_title']}"))
    story.append(p(f"Client: {SUBMISSION['client_name']}"))
    story.append(p(f"Project Root: {PROJECT_ROOT}"))
    story.append(p(f"Submission Date: {SUBMISSION['submission_date']}"))
    story.append(Spacer(1, 12))

    story.append(Paragraph("Executive Summary", styles["Heading2"]))
    story.append(
        p(
            "This submission presents an end-to-end recommendation data pipeline for RecoMart. "
            "The implementation covers problem formulation, multi-source ingestion, raw data storage, "
            "data validation with remediation and revalidation, preparation, feature engineering, "
            "a custom feature store, model tracking, and pipeline orchestration."
        )
    )
    story.extend(bullets(SUBMISSION["objectives"]))

    # 1
    story.append(Paragraph("1. Problem Formulation", styles["Heading2"]))
    story.append(p(SUBMISSION["business_problem"]))
    story.append(Paragraph("Expected Outputs", styles["Heading3"]))
    story.extend(
        bullets(
            [
                "Clean datasets for EDA.",
                "Engineered features for collaborative/content-based recommendation models.",
                "Deployable recommendation model artifacts and inference-ready datasets.",
                "Operational logs, validation reports, and tracked training metadata.",
            ]
        )
    )
    story.append(Paragraph("Evaluation Metrics", styles["Heading3"]))
    story.extend(bullets(SUBMISSION["evaluation_metrics"]))

    # 2
    story.append(Paragraph("2. Data Collection and Ingestion", styles["Heading2"]))
    story.append(
        p(
            "The pipeline ingests at least two data types, satisfying the assignment requirement: "
            "CSV-based RetailRocket interaction/catalog data and JSON-based DummyJSON product/category data."
        )
    )
    ingestion_rows = [["Source Key", "Exact Path", "Exists"]]
    for k, v in DISCOVERED_FILES.items():
        ingestion_rows.append([k, str(v), "Yes" if v.exists() else "No"])
    add_table(story, "Discovered Ingestion Files", ingestion_rows, col_widths=[4*cm, 10*cm, 2*cm])

    # 3
    story.append(Paragraph("3. Raw Data Storage", styles["Heading2"]))
    storage_rows = [
        ["Folder", "Purpose", "Exists"],
        [str(RAW_ROOT), "Raw source data", "Yes" if RAW_ROOT.exists() else "No"],
        [str(BRONZE_ROOT), "Structured bronze datasets", "Yes" if BRONZE_ROOT.exists() else "No"],
        [str(REMEDIATED_RAW_ROOT), "Remediated raw outputs", "Yes" if REMEDIATED_RAW_ROOT.exists() else "No"],
        [str(REMEDIATED_BRONZE_ROOT), "Remediated bronze outputs", "Yes" if REMEDIATED_BRONZE_ROOT.exists() else "No"],
        [str(REPORT_ROOT), "Validation reports", "Yes" if REPORT_ROOT.exists() else "No"],
        [str(LOG_ROOT), "Execution logs", "Yes" if LOG_ROOT.exists() else "No"],
    ]
    add_table(story, "Exact Storage Layout", storage_rows, col_widths=[8*cm, 6*cm, 2*cm])
```

```
    # 4
    story.append(Paragraph("4. Data Profiling and Validation", styles["Heading2"]))
    story.append(
        p(
            "The validation stage includes robust project-root discovery, issue logging, remediation, "
            "revalidation, and PDF/JSON/CSV report generation."
        )
    )
    validation_rows = [
        ["Field", "Value"],
        ["Validation Run ID", VALIDATION_RUN_ID],
        ["Initial Status", VALIDATION_INITIAL_STATUS],
        ["Final Status", VALIDATION_FINAL_STATUS],
        ["JSON Report", str(VALIDATION_ARTIFACTS["json_report"])],
        ["PDF Report", str(VALIDATION_ARTIFACTS["pdf_report"])],
        ["Fix Log", str(VALIDATION_ARTIFACTS["fix_log"])],
    ]
    add_table(story, "Validation Run Summary", validation_rows, col_widths=[5*cm, 11*cm])

    artifact_rows = [["Artifact", "Path", "Exists"]]
    for k, v in VALIDATION_ARTIFACTS.items():
        artifact_rows.append([k, str(v), "Yes" if v.exists() else "No"])
    add_table(story, "Validation Artifacts", artifact_rows, col_widths=[4*cm, 10*cm, 2*cm])

    # 5
    story.append(Paragraph("5. Data Preparation", styles["Heading2"]))
    story.append(
        p(
            "Prepared datasets are created after cleaning and remediation. Based on the validation pipeline, "
            "this includes fixing problematic raw and bronze files and carrying forward standardized datasets "
            "for downstream use."
        )
    )
    story.extend(
        bullets(
            [
                "Missing/invalid values addressed during remediation.",
                "Dataset consistency rechecked in revalidation.",
                "Prepared outputs routed into remediated/raw and remediated/bronze folders.",
            ]
        )
    )

    # 6
    story.append(Paragraph("6. Feature Engineering and Transformation", styles["Heading2"]))
    story.append(
        p(
            "Feature engineering logic is expected to transform prepared recommendation data into reusable "
            "training and inference features. The repository structure indicates a dedicated feature stage "
            "under src and a separate feature_store area for metadata-based retrieval."
        )
    )
    story.extend(
        bullets(
            [
                "Recommendation-oriented transformation stage exists under src.",
                "Feature outputs are intended to be version-aware and retrievable for training/inference.",
                "Supporting code is included in the appendix automatically from src/ and feature_store/.",
            ]
        )
    )

    # 7
    story.append(Paragraph("7. Feature Store", styles["Heading2"]))
    feature_registry = FEATURE_STORE_ROOT / "registry" / "feature_registry.yaml"
    feature_rows = [
        ["Item", "Value"],
        ["Feature Store Root", str(FEATURE_STORE_ROOT)],
        ["Registry File", str(feature_registry)],
        ["Registry Present", "Yes" if feature_registry.exists() else "No"],
    ]
    add_table(story, "Feature Store Metadata", feature_rows, col_widths=[4*cm, 12*cm])

    # 8
    story.append(Paragraph("8. Data Versioning and Lineage", styles["Heading2"]))
    story.extend(
        bullets(
            [
                "Raw data is partitioned by source plus timestamp folders such as load_date and load_hour.",
                "Validation outputs are versioned by run I

[TRUNCATED FOR PDF SIZE]
```

Appendix B: Final Remarks

This personalized report uses exact project paths and the exact validation run evidence already captured in the repository. If MLflow or Prefect metadata folders are present locally, they will be pulled into the final PDF automatically. Before submission, fill in the roll number and verify that the generated PDF, source code, video walkthrough, and zip package are all included.